

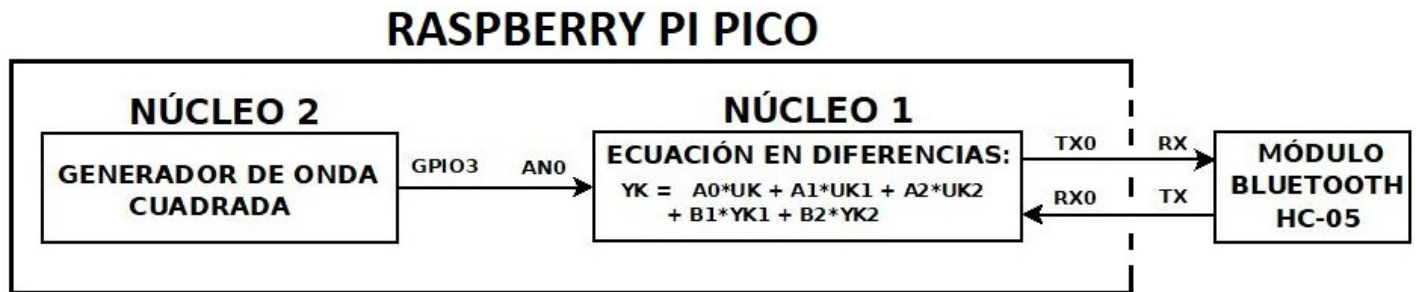
SIMULACIÓN DE SISTEMAS DISCRETOS DE PRIMERO Y SEGUNDO ORDEN MEDIANTE EL USO DE UNA RASPBERRY PI PICO UTILIZANDO THREADS

M. en I. José Antonio de Jesús Arredondo Garza

En este proyecto se está presentando un sistema basado en dos núcleos, en donde el primer núcleo implementa un esquema de ecuación en diferencias y el segundo núcleo está destinado a generar una onda cuadrada cuyo periodo puede ser programable. Como apoyo al proyecto se usará una interfaz basada en el “*appdesigner*” de MATLAB, de la cual, se hará una amplia mención en el apéndice “C”.

La ecuación en diferencias proveniente de una función de transferencia discreta se planteó de manera que sus coeficientes puedan ser modificados para simular sistemas de primero y segundo orden.

La frecuencia de muestreo es a una **tasa fija de 8 Khz**, debido a esto, todos los sistemas deben considerar esta frecuencia para su implementación. Para la transmisión/recepción de datos se está usando un módulo bluetooth HC-05 que es muy económico, el cual, está configurado para manejar una tasa de baudaje de 460800 bauds. En el apéndice “A” se hará una breve descripción sobre como configurar el HC-05. Finalmente, en el apéndice “B” se presentará en diagrama de flujo del programa desarrollado en lenguaje Micropython para la Raspberry Pi Pico usando los 2 núcleos.



NOTA: el Pin GPIO3 (onda cuadrada), deberá estar conectado al Pin GPIO26 (canal AN0 del convertidor A/D).

La función de transferencia de segundo orden discreta planteada sería de la forma:

$$H(z) = \frac{A0 * Z^2 + A1 * Z + A2}{Z^2 + B1 * Z + B2} = \frac{Y(z)}{U(z)}$$

Para obtener la ecuación en diferencias necesitamos dividir entre Z^2 , con lo cual obtenemos:

$$H(z) = \frac{A0 * Z^0 + A1 * Z^{-1} + A2 * Z^{-2}}{Z^0 + B1 * Z^{-1} + B2 * Z^{-2}} = \frac{Y(z)}{U(z)}$$

Descomponiendo la función de transferencia en términos de entradas y salidas se tiene:

$$Y(z)[Z^0 + B1 * Z^{-1} + B2 * Z^{-2}] = U(z)[A0 * Z^0 + A1 * Z^{-1} + A2 * Z^{-2}]$$

Para obtener las anti transformadas correspondientes hay que tomar en cuenta:

$f(z) * Z^0$	$\leftrightarrow$	$f(k)$	Evento actual
$f(z) * Z^{-n}$	$\leftrightarrow$	$f(k - 1)$	Eventos pasados
$f(z) * Z^{+n}$	$\leftrightarrow$	$f(k + 1)$	Eventos futuros

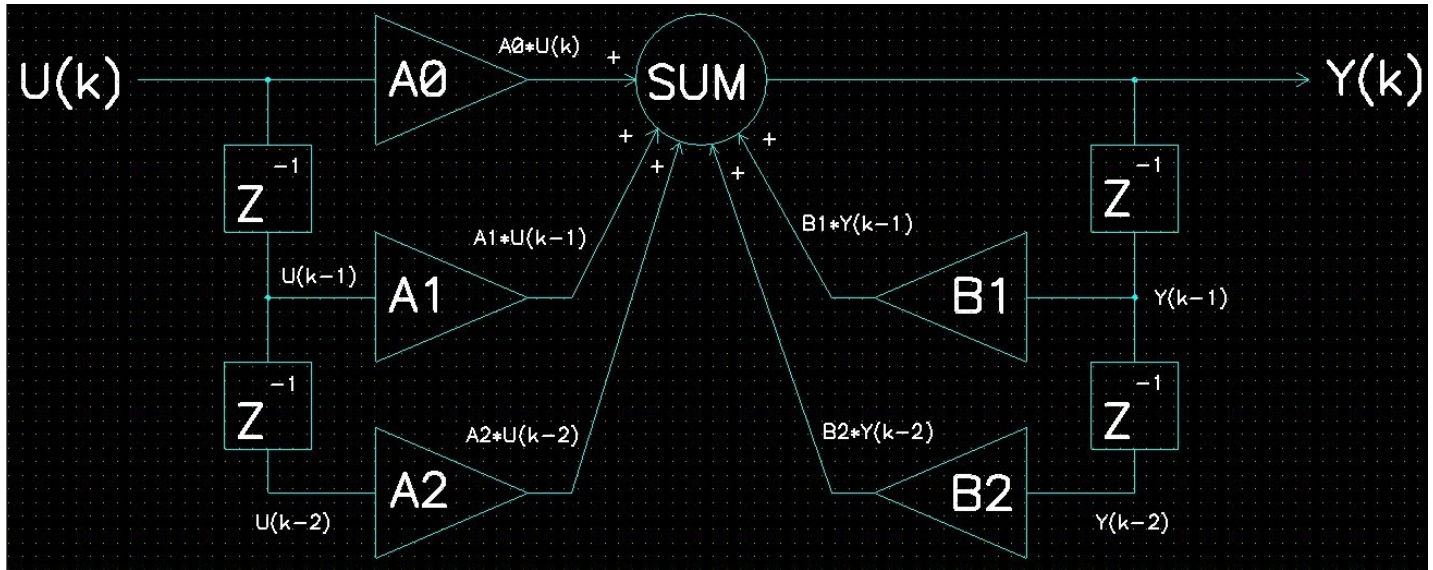
Tomando en cuenta lo anterior se tiene entonces:

$$Y(k) + B1 * Y(k - 1) + B2 * y(k - 2) = A0 * U(k) + A1 * U(k - 2) + A2 * U(k - 2)$$

Despejando la salida actual se obtiene entonces la ecuación en diferencias:

$$Y(k) = A0 * U(k) + A1 * U(k - 2) + A2 * U(k - 2) - B1 * Y(k - 1) - B2 * Y(k - 2)$$

El diagrama de flujo correspondiente:



Aproximadamente en el año de 1976 muy previo a la aparición de los Procesadores Digitales de Señales “**DSP**” (año de 1982), la compañía “*Advance Micro Devices*” lanzó al mercado unas compuertas muy especializadas para implementar este tipo de esquemas, consistentes en “*multiplicadores paralelo – serie*”, “*sumadores seriales*” y para crear los retardos Z^{-1} se emplearon “*flip – flops tipo D*”. Toda esta tarea se realizaba en forma de datos serie (*strings*) y utilizaron notación aritmética en punto fijo. Es muy importante hacer notar que la aritmética inicial de los DSP’s desarrollados por “*Texas Instruments*” fue en punto fijo.

EJEMPLO1. Supongamos la siguiente función de transferencia que representa a un sistema subamortiguado (vea el archivo *LRC_Suba_UART0.py*):

$$H(z) = \frac{0.03604Z^2 + 0.07207Z + 0.03604}{Z^2 - 1.774Z + 0.9177}$$

Desnormalizando la función de transferencia:

$$H(z) = \frac{0.03604Z^0 + 0.07207Z^{-1} + 0.03604Z^{-2}}{Z^0 - 1.774Z^{-1} + 0.9177Z^{-2}} = \frac{Y(z)}{U(z)}$$

En función de salidas y entradas:

$$Y(z)[Z^0 - 1.774Z^{-1} + 0.9177Z^{-2}] = U(z)[0.03604Z^0 + 0.07207Z^{-1} + 0.03604Z^{-2}]$$

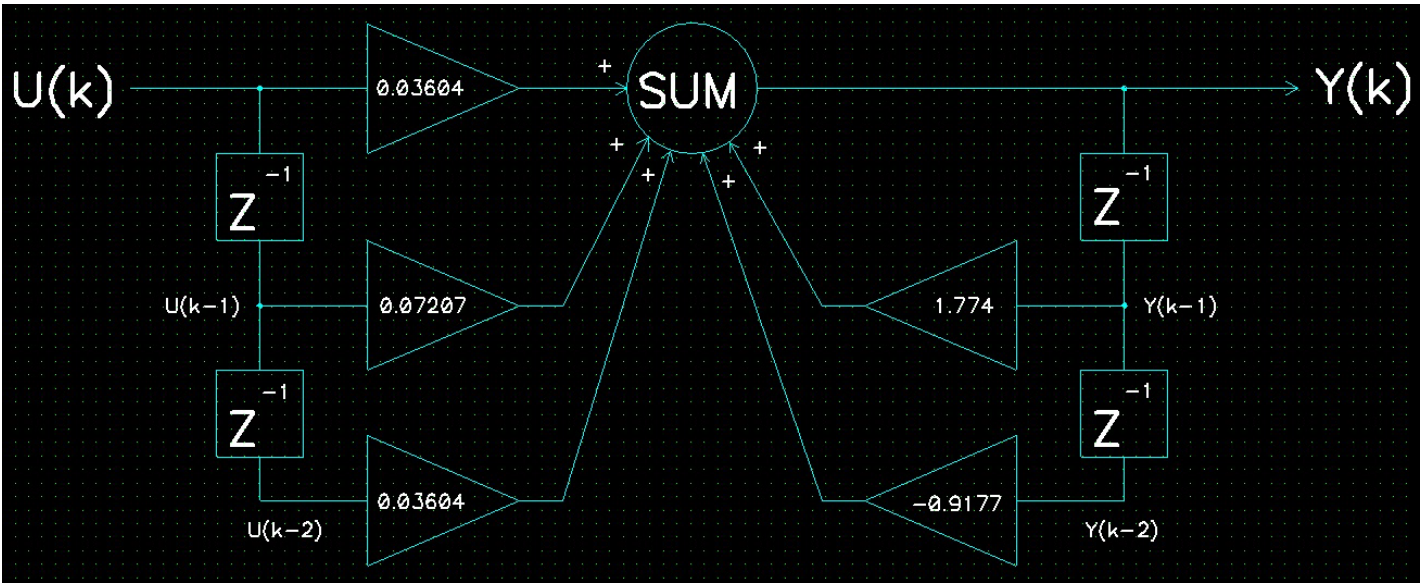
Antitransformando:

$$Y(k) - 1.774Y(k - 1) + 0.9177Y(k - 2) = 0.03604U(k) + 0.07207U(k - 1) + 0.03604U(k - 2)$$

La ecuación en diferencias resultante es:

$$Y(k) = 0.03604U(k) + 0.07207U(k - 1) + 0.03604U(k - 2) + 1.774Y(k - 1) - 0.9177Y(k - 2)$$

El diagrama de flujo sería:



El programa en un script de MATLAB que será muy parecido a la programación en la Raspberry Pi Pico es:

```
% =====
%
%   SIMULACIÓN DE LA ECUACIÓN EN DIFERENCIAS
%
% =====

A0 = 0.03604;
A1 = 0.07207;
A2 = 0.03604;
B1 = 1.774;
B2 = -0.9177;

UK = 0;      % U(k) = 0
UK1 = 0;     % U(k-1) = 0
UK2 = 0;     % U(k-2) = 0
YK = 0;      % Y(k) = 0
YK1 = 0;     % Y(k-1) = 0
YK2 = 0;     % Y(k-2) = 0

Escalon = 1.0; % Simula un escalón

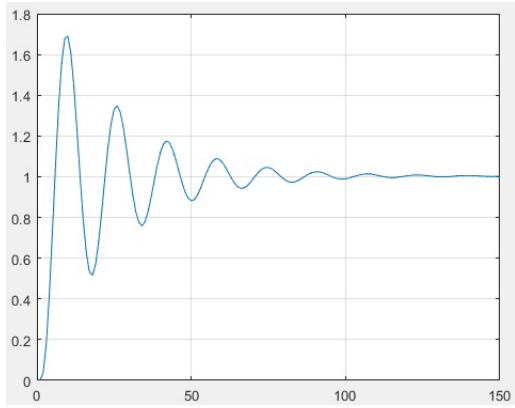
% Loop de Iteraciones
for i = 1:150
    % La ecuación en diferencias:
    YK = A0*UK + A1*UK1 + A2*UK2 + B1*YK1 + B2*YK2;

    % Colecta soluciones:
    Sol(i) = YK;

    % Actualiza Salidas y Entradas para la próxima iteración:
    UK2 = UK1; % U(k-1) ----> U(k-2)
    UK1 = UK;  % U(k)   ----> U(k-1)
    YK2 = YK1; % Y(k-1) ----> Y(k-2)
    YK1 = YK;  % Y(k)   ----> Y(k-1)

    % Asigna la nueva U(k):
    UK = Escalon;
end
plot(Sol) % Grafica las soluciones.
```

La Respuesta a Escalón para este sistema es:



La gráfica en tiempo real obtenida es:



EJEMPLO 2. Supongamos ahora el caso de un sistema de primer orden adaptado al formato de la ecuación en diferencias de segundo orden (corresponde al archivo `RC_UART0.py`):

$$H(z) = \frac{0.0303Z + 0.0303}{Z - 0.9394}$$

Desnormalizando:

$$H(z) = \frac{0.0303Z^0 + 0.0303Z^{-1}}{Z^0 - 0.9394Z^{-1}} = \frac{Y(z)}{U(z)}$$

En función de salidas y entradas:

$$Y(z)[Z^0 - 0.9394Z^{-1}] = U(z)[0.0303Z^0 + 0.0303Z^{-1}]$$

Antitransformando:

$$Y(k) - 0.9394Y(k-1) = 0.0303U(k) + 0.0303U(k-1)$$

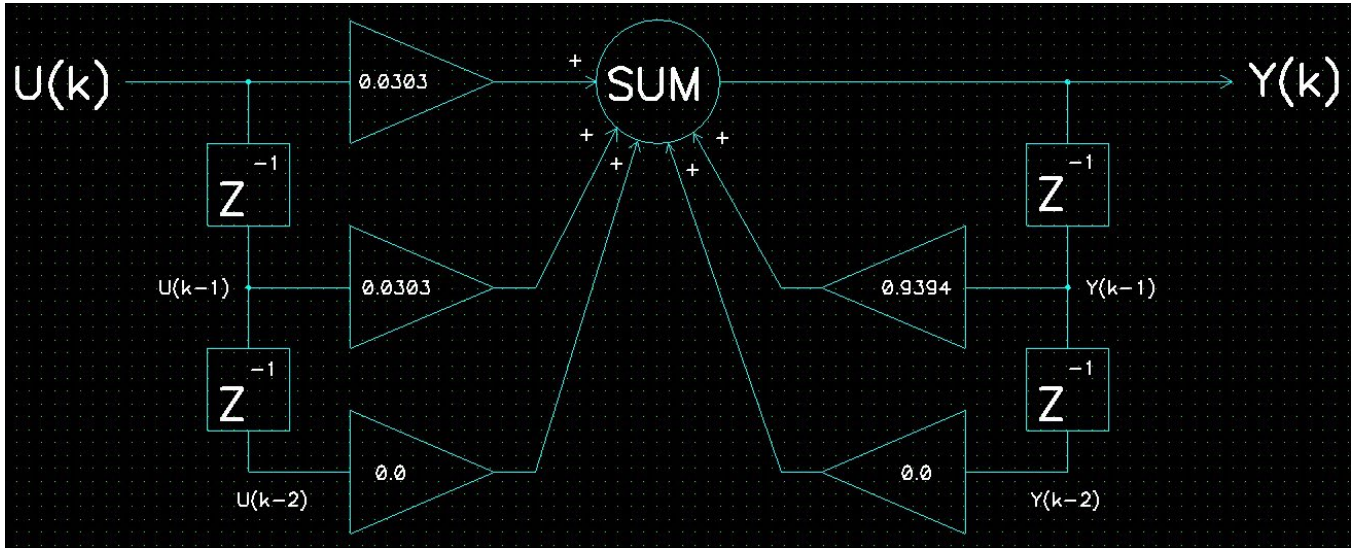
La ecuación en diferencias es:

$$Y(k) = 0.0303U(k) + 0.0303U(k-1) + 0.9394Y(k)$$

Adaptando al formato de la ecuación en diferencias de segundo orden:

$$Y(k) = 0.0303U(k) + 0.0303U(k-1) + 0 * U(k-2) + 0.9394Y(k-1) + 0 * Y(k-2)$$

EL diagrama de flujo:



Aunque se podrían eliminar los coeficientes “A2” y “B2”, así como la entrada $U(k-2)$ y $Y(k-2)$ en el script de MATLAB, se dejando estos términos para ejemplificar lo que está pasando en el programa en Python de la Raspberry Pi Pico “RP2”. Se tomó la decisión de dejarlos en el programa por cuestiones de compatibilidad en el retardo inherente a los tiempos de ejecución de las instrucciones.

Para lograr los 125 uSeg (8 KHz de frecuencia de muestreo), se vio experimentalmente que programa consume 59 uSeg y para completar el retardo faltante se usaron 66 uSeg extras ($59 + 66 = 125$ uSeg). Mas adelante se hará una mención al respecto en el programa en Python de la RP2.

```
% =====
%
%  SIMULACIÓN DE LA ECUACIÓN ENDIFERENCIAS
%
% =====

A0 = 0.0303;
A1 = 0.0303;
A2 = 0.0;
B1 = 0.9394;
B2 = 0.0;

UK = 0;      % U(k) = 0
UK1 = 0;     % U(k-1) = 0
UK2 = 0;     % U(k-2) = 0
YK = 0;      % Y(k) = 0
YK1 = 0;     % Y(k-1) = 0
YK2 = 0;     % Y(k-2) = 0

Escalon = 1.0; % Simula un escalón

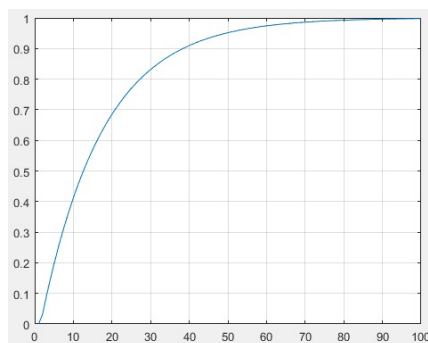
% Loop de Iteraciones
for i = 1:110
    % La ecuación en diferencias:
    YK = A0*UK + A1*UK1 + A2*UK2 + B1*YK1 + B2*YK2;

    % Colecta soluciones:
    Sol(i) = YK;

    % Actualiza Salidas y Entradas para la próxima iteración:
    UK2 = UK1;    % U(k-1) ---> U(k-2)
    UK1 = UK;     % U(k)   ---> U(k-1)
    YK2 = YK1;    % Y(k-1) ---> Y(k-2)
    YK1 = YK;     % Y(k)   ---> Y(k-1)

    % Asigna la nueva U(k):
    UK = Escalon;
end
plot(Sol)      % Grafica las soluciones.
grid on
```

La respuesta a escalón para este segundo ejemplo es:

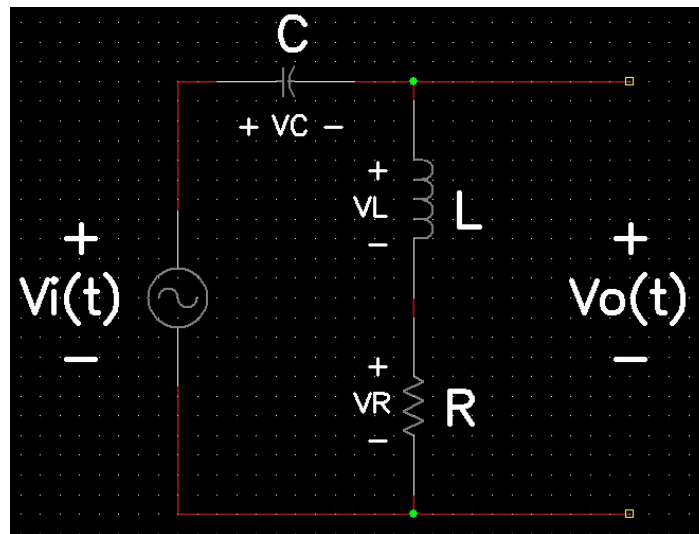


La gráfica en tiempo real obtenida es:



EJEMPLO 3. Simulación de un sistema subamortiguado basado en un circuito “C_LR” como el mostrado en la figura de abajo, en donde la frecuencia real de oscilación es $f = 500 \text{ Hz}$ y el sobrepaso máximo es $M_p = 70\%$ (corresponde al archivo `C__LR_Subamortiguado.py`).

NOTA: este ejemplo muy polémico sólo responde a un caso de implementación en tiempo real.



Leyes de Elemento:

$$i_C = C \frac{dv_C}{dt} \dots 1$$

$$V_L = L \frac{di_L}{dt} \dots 2$$

$$V_R = Ri_R \dots 3$$

Leyes de Kirchoff de Voltajes y de Corrientes:

$$\sum V's = 0 \dots 4$$

$$\sum i's = 0 \dots 5$$

Las ecuaciones de voltajes son:

$$V_i(t) = V_C + V_L + V_R \dots 6$$

$$V_L + V_R = V_o(t) \dots 7$$

La ecuación de corrientes de malla:

$$i_C = i_L = i_R \dots 8$$

Sustituyendo 7 en 6:

$$V_i(t) = V_C + V_o(t) \dots 9$$

Sustituyendo la Ley Inversa del Capacitor de la Ecuación1:

$$V_i(t) = \left(\frac{1}{C}\right) \int_0^t i_C dt + V_o(t) \dots 10$$

Para despejar el valor de i_C se tiene:

$$\left(\frac{1}{C}\right) \int_0^t i_C dt = V_i(t) - V_o(t) \dots 11$$

$$i_C = C \frac{d}{dt} [V_i(t) - V_o(t)] \dots 12$$

De la ecuación 7 se tiene:

$$L \frac{di_L}{dt} + Ri_R = V_o(t) \dots 13$$

Sustituyendo la ecuación de corrientes de 12 en 13:

$$L \frac{d}{dt} \left\{ C \frac{d}{dt} [V_i(t) - V_o(t)] \right\} + R \left\{ C \frac{d}{dt} [V_i(t) - V_o(t)] \right\} = V_o(t) \dots 14$$

Desarrollando y factorizando:

$$LC \frac{d^2 V_i(t)}{dt^2} - LC \frac{d^2 V_o(t)}{dt^2} + RC \frac{dV_i(t)}{dt} - RC \frac{dV_o(t)}{dt} = V_o(t) \dots 15$$

$$LC \frac{d^2 V_o(t)}{dt^2} + RC \frac{dV_o(t)}{dt} + V_o(t) = LC \frac{d^2 V_i(t)}{dt^2} + RC \frac{dV_i(t)}{dt} \dots 16$$

Finalmente, el modelo matemático en función de $V_o(t)$:

$$\frac{d^2 V_o(t)}{dt^2} + \left(\frac{R}{L}\right) \frac{dV_o(t)}{dt} + \left(\frac{1}{LC}\right) V_o(t) = \frac{d^2 V_i(t)}{dt^2} + \left(\frac{R}{L}\right) \frac{dV_i(t)}{dt} \dots 17$$

Transformado y asumiendo las constantes asociadas a la transformación iguales a cero:

$$S^2 V_o(s) + \left(\frac{R}{L}\right) S V_o(s) + \left(\frac{1}{LC}\right) V_o(s) = S^2 V_i(s) + \left(\frac{R}{L}\right) S V_i(s) \dots 18$$

$$\left[S^2 + \left(\frac{R}{L} \right) S + \left(\frac{1}{LC} \right) \right] V_o(s) = \left[S^2 + \left(\frac{R}{L} \right) S \right] V_i(s) \dots 19$$

La Función de Transferencia resultante es:

$$H(s) = \frac{S^2 + \left(\frac{R}{L} \right) S}{S^2 + \left(\frac{R}{L} \right) S + \left(\frac{1}{LC} \right)} = \frac{V_o(s)}{V_i(s)} \dots 20$$

Con la información proporcionada, la frecuencia real de oscilación en términos de ω es:

$$\omega = 2\pi f = 2 * \pi * 500 = 3141.592651 \left[\frac{rad}{seg} \right] \dots 21$$

Para calcular la frecuencia natural de oscilación se necesita calcular el coeficiente de amortiguamiento ξ , por lo que, tomando en cuenta el sobrepaso máximo como dato se tiene:

$$M_P = 70\% = e^{-\frac{\pi\xi}{\sqrt{1-\xi^2}}} \times 100 \dots 22$$

$$0.7 = e^{-\frac{\pi\xi}{\sqrt{1-\xi^2}}} \dots 23$$

Proceso de despeje y cálculo del coeficiente de amortiguamiento ξ :

$$\ln(0.7) = -\frac{\pi\xi}{\sqrt{1-\xi^2}} \dots 24$$

$$-0.35667494 = -\frac{\pi\xi}{\sqrt{1-\xi^2}} \dots 25$$

$$(-0.35667494)\sqrt{1-\xi^2} = -\pi\xi \dots 26$$

$$(0.35667494)\sqrt{1-\xi^2} = \pi\xi \dots 27$$

Elevando al cuadrado:

$$(0.1272170128)(1-\xi^2) = \pi^2\xi^2 \dots 28$$

Agrupando términos semejantes:

$$0.1272170128 = \pi^2\xi^2 + 0.1272170128\xi^2 \dots 29$$

$$0.1272170128 = \xi^2(\pi^2 + 0.1272170128) \dots 30$$

$$\xi^2 = \frac{0.1272170128}{\pi^2 + 0.1272170128} \dots 31$$

Finalmente, el valor del coeficiente de amortiguamiento ξ es:

$$\xi = \sqrt{\frac{0.1272170128}{\pi^2 + 0.1272170128}} = \sqrt{\frac{0.1272170128}{9.9968214138893}} = 0.112808449453 \dots 32$$

Para el cálculo de la frecuencia real de oscilación ω_o se tiene:

$$\omega = \omega_o \sqrt{1 - \xi^2} \dots 33$$

$$\omega_o = \frac{\omega}{\sqrt{1 - \xi^2}} = 3161.775039 \left[\frac{\text{rad}}{\text{seg}} \right] \dots 34$$

Obligando a la función de transferencia $H(s)$ a ser subamortiguada se tiene:

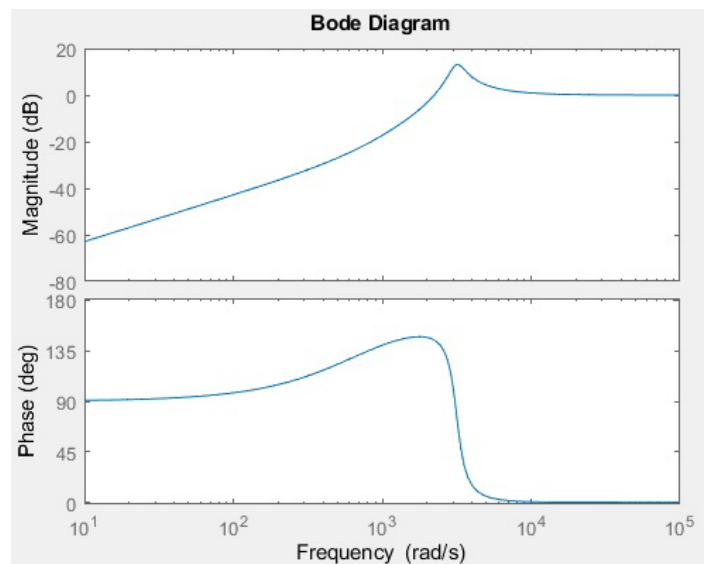
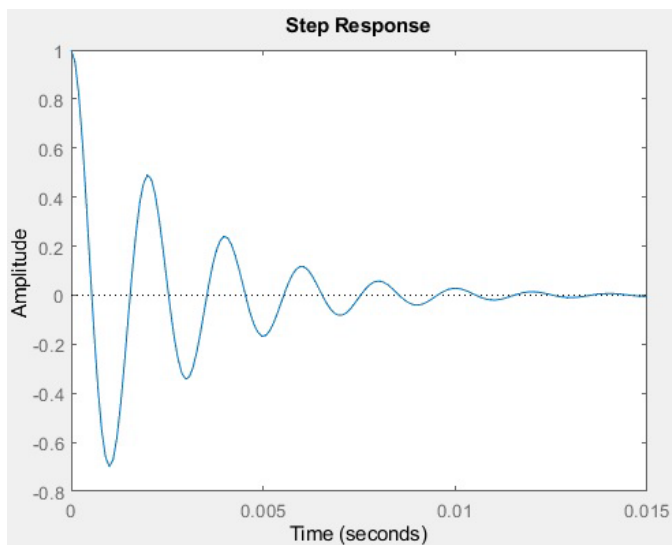
$$H(s) = \frac{s^2 + \left(\frac{R}{L}\right)s}{s^2 + \left(\frac{R}{L}\right)s + \left(\frac{1}{LC}\right)} = \frac{s^2 + 2\xi\omega_o s}{s^2 + 2\xi\omega_o s + \omega_o^2} = \frac{V_o(s)}{V_i(s)} \dots 35$$

Sustituyendo valores:

$$H(s) = \frac{s^2 + 713.3499s}{s^2 + 713.3499s + 9996821.4167377} \dots 36$$

Probando la respuesta a escalón y su respuesta en frecuencia usando MATLAB es:

```
>> Sys = tf([1 713.3499 0],[1 713.3499 9996821.4167377]);
>> step(Sys)
>> bode(Sys)
```



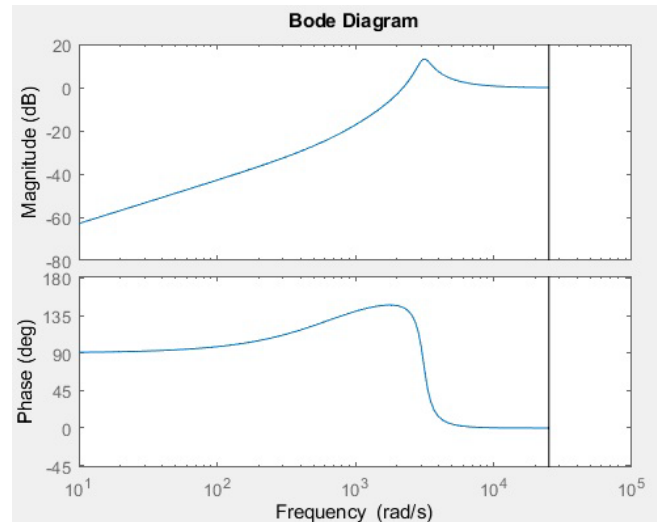
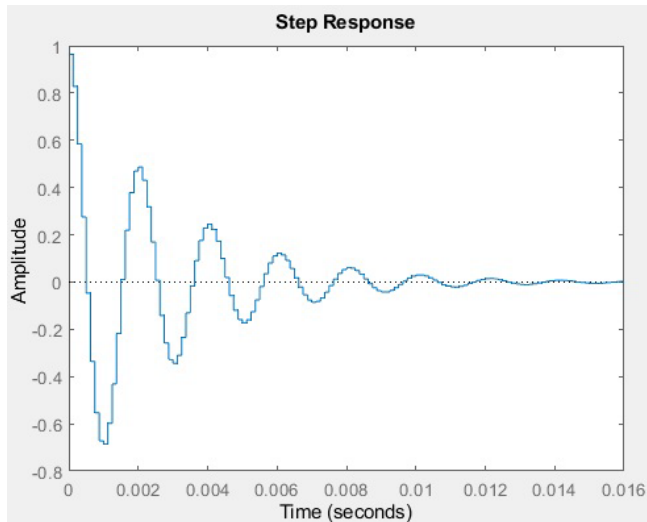
Para convertir la función de transferencia continua $H(s)$ a una función de transferencia discreta $H(z)$ utilizando el método de "Tustin" con una frecuencia de muestreo $f_s = 8000 \text{ Hz}$ (que equivale a $T_s = \frac{1}{f_s} = 125 \mu\text{Seg}$), se tiene:

```
>> Sysd = c2d(Sys, 0.000125, 'tustin')
```

$$H(z) = \frac{0.964Z^2 - 1.846Z + 0.8817}{Z^2 - 1.774Z + 0.9177} \dots 37$$

La respuesta a escalón discreto y su diagrama de Bode:

```
>> step(Sysd)
>> bode(Sysd)
```



Para obtener la ecuación en diferencias primero dividimos numerador y denominador entre Z^2 :

$$H(z) = \frac{0.964 - 1.846Z^{-1} + 0.8817Z^{-2}}{1 - 1.774Z^{-1} + 0.9177Z^{-2}} = \frac{Y(z)}{U(z)} \dots 38$$

Descomponiendo en términos de sus salidas y entradas:

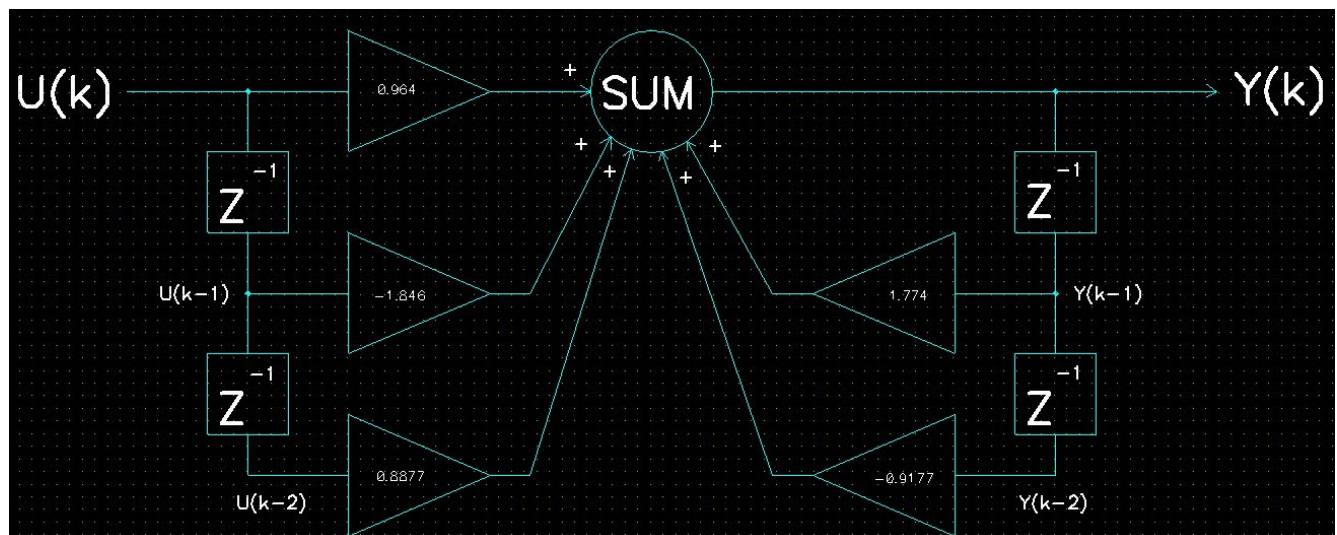
$$Y(z)[1 - 1.774Z^{-1} + 0.9177Z^{-2}] = U(z)[0.964 - 1.846Z^{-1} + 0.8817Z^{-2}] \dots 39$$

Antitransformando se obtiene:

$$Y(k) - 1.774Y(k-1) + 0.9177Y(k-2) = 0.964U(k) - 1.846U(k-1) + 0.8877U(k-2) \dots 40$$

Finalmente, la ecuación diferencias es:

$$Y(k) = 0.964U(k) - 1.846U(k-1) + 0.8877U(k-2) + 1.774Y(k-1) - 0.9177Y(k-2) \dots 41$$



El "script" de MATLAB es:

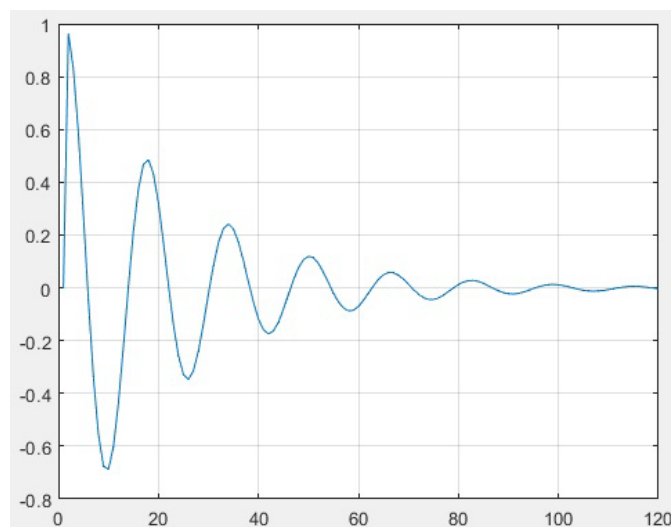
```
% ===== %
% SIMULACIÓN EN MATLAB DE LA ECUACIÓN EN DIFERENCIAS "C_LR" %
% ===== %

% Los Coeficientes:
CTE_A0 = +0.964;
CTE_A1 = -1.846;
CTE_A2 = +0.8817;
CTE_B1 = +1.774;
CTE_B2 = -0.9177;

% Declaración de Entradas/Salidas:
UK = 0; % U(k) = 0
UK1 = 0; % U(k-1) = 0
UK2 = 0; % U(k-2) = 0
YK = 0; % Y(k) = 0
YK1 = 0; % Y(k-1) = 0
YK2 = 0; % Y(k-2) = 0

% Simulación del Escalón de Entrada:
ESCALON = 1.0;
% Loop de Iteraciones:
for i = 1:100
    % La ecuación en diferencias:
    YK = CTE_A0*UK + CTE_A1*UK1 + CTE_A2*UK2 + CTE_B1*YK1 + CTE_B2*YK2;
    % Colecta soluciones:
    GRAPH(i) = YK;

    % Actualiza Entradas/Salidas:
    UK2 = UK1; % U(k-1) --> U(k-2)
    UK1 = UK; % U(k) --> U(k-1)
    YK2 = YK1; % Y(k-1) --> Y(k-2)
    YK1 = YK; % Y(k) --> Y(k-1)
    UK = ESCALON; % Asigna la nueva U(k)
end
plot(GRAPH)
grid on
```



La respuesta del circuito “C_LR” subamortiguado en tiempo real es la siguiente:



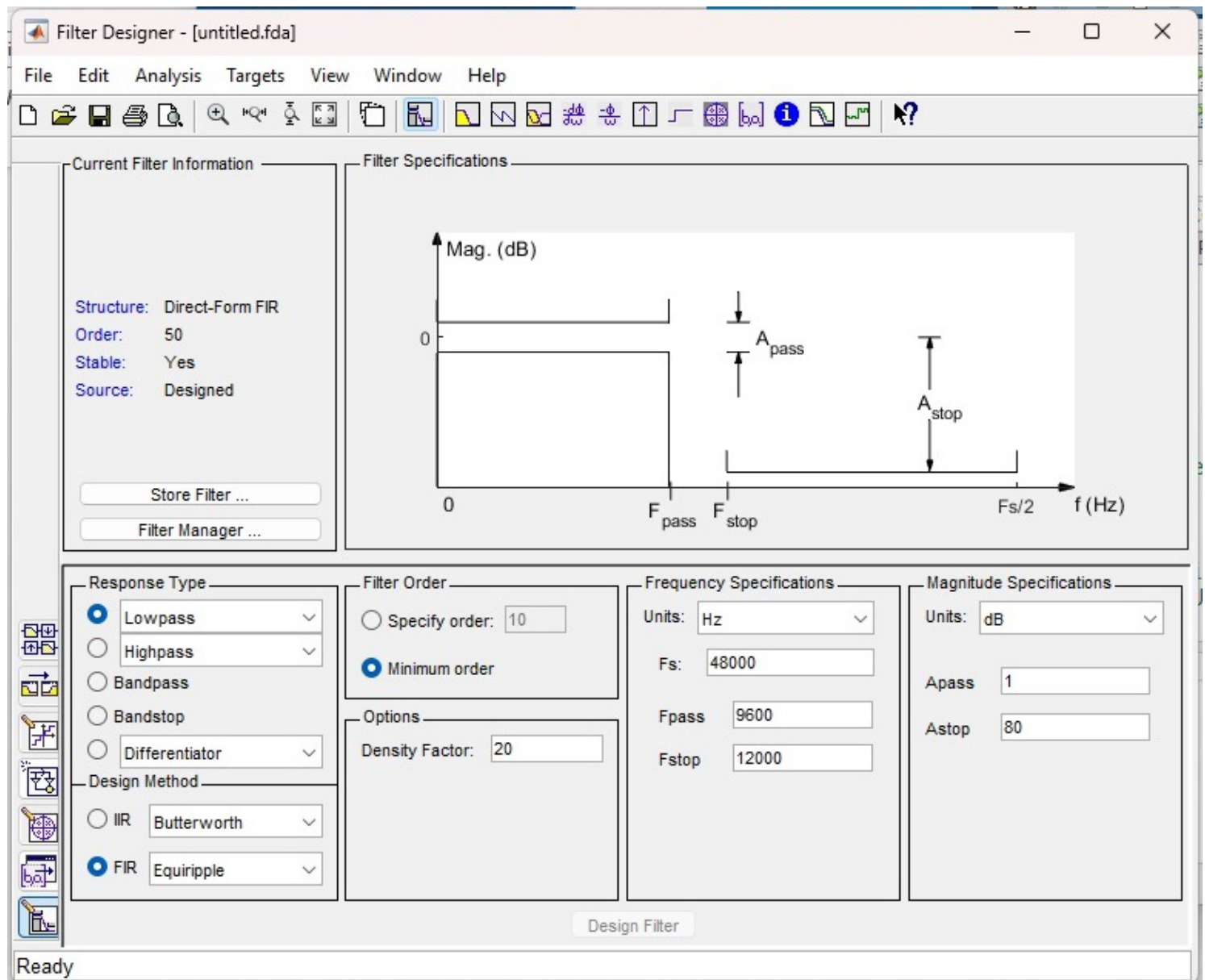
USO DE LA HERRAMIENTA PARA DISEÑO DE FILTROS DIGITALES DE MATLAB

EJEMPLO 4. Implementación de un Filtro IIR (Infinite Impulse Response) de estructura Butterworth de segundo orden pasa banda, con una frecuencia $f_o = 400 \text{ Hz}$ y un ancho de banda $BW = 20 \text{ Hz}$. Por supuesto, la frecuencia de muestreo seguirá siendo de 8 KHz. Se usará la herramienta “*fdatool*” de MATLAB, lo cual, permitirá hacer un diseño por plantilla. A continuación, se enumerarán los pasos que se deben seguir para el diseño del filtro (vea el archivo *BandPass_400Hz/BandPass_400Hz.py*).

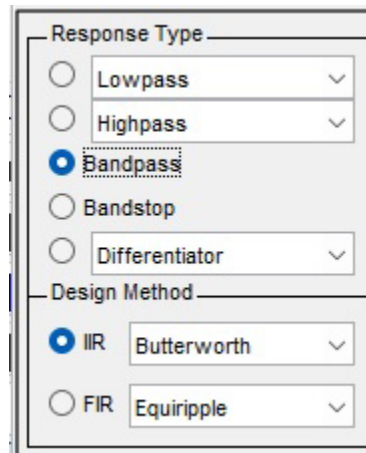
1. Abra el programa MATLAB y en la línea de comando escriba “*fdatool*”.



2. En seguida se estará abriendo el programa, apareciendo inmediatamente el “*tip del día*”. Aparecerá la siguiente pantalla:

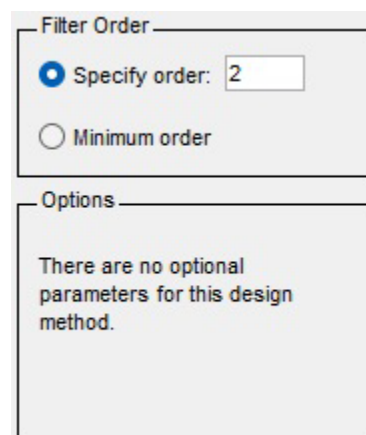


3. Para configurar la respuesta del filtro deberá seleccionar la configuración “**IIR**” y “**Bandpass**” como se observa en la figura. **Nota:** la sesión puede ser salvada como proyecto.



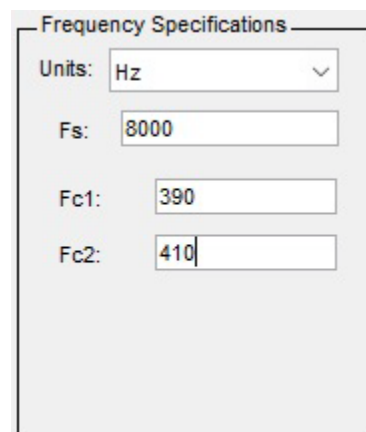
The screenshot shows the 'Response Type' and 'Design Method' sections of a filter design interface. Under 'Response Type', the 'Bandpass' option is selected with a radio button. Under 'Design Method', the 'IIR' option is selected with a radio button, and the 'Butterworth' filter type is chosen from the dropdown menu.

4. Para lograr especificar el orden del filtro se deberá hacer lo siguiente:



The screenshot shows the 'Filter Order' and 'Options' sections. In the 'Filter Order' section, the 'Specify order' radio button is selected, and the value '2' is entered in the adjacent text box. The 'Options' section below contains the text: 'There are no optional parameters for this design method.'

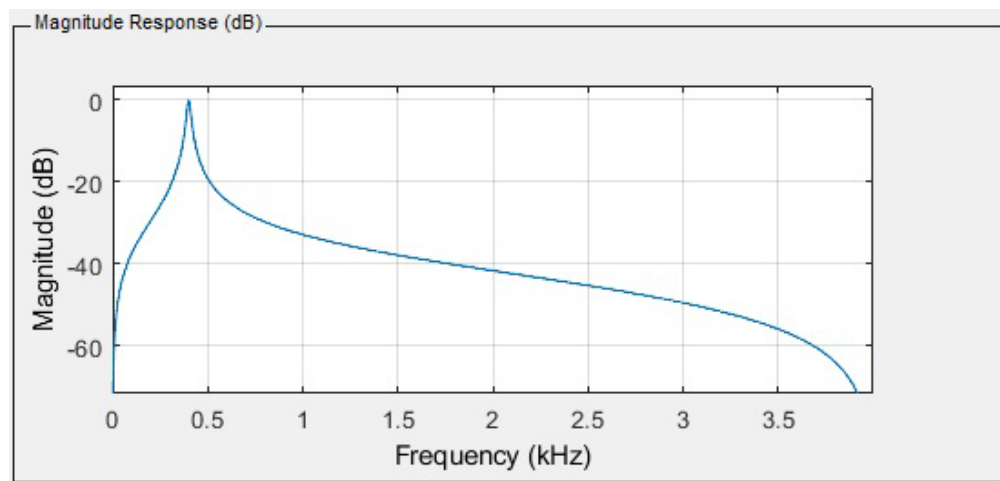
5. A continuación, en campo de la frecuencia de muestreo se declarará como $F_s = 8000\text{Hz}$ y el ancho de banda se especificará como $F_{C1} = 390\text{Hz}$ y $F_{C2} = 410\text{Hz}$ para tener un ancho de banda $BW = F_{C2} - F_{C1}$ que será igual a 20 Hz.



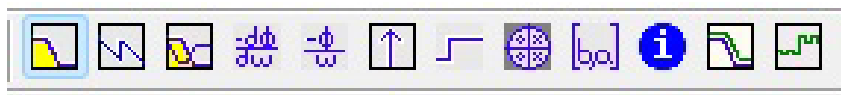
The screenshot shows the 'Frequency Specifications' section. The 'Units' dropdown is set to 'Hz'. The 'Fs' (sampling frequency) is set to 8000. The 'Fc1' (lower cutoff frequency) is set to 390, and the 'Fc2' (upper cutoff frequency) is set to 410.

6. Una vez que se tienen asignados todos los parámetros, se procede a realizar el diseño del filtro. Esto se logra pulsando el botón “Design Filter”.

7. En seguida aparecerá la gráfica de respuesta en frecuencia del filtro en donde se ve que checa con los parámetros asignados ($f_o = 400\text{Hz}$).



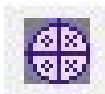
8. A continuación, hay unos íconos que nos van a ser muy útiles para llegar a la implementación, los cuales son:



Gráfica de Respuesta en Frecuencia



Gráfica de la Fase y Respuesta en Frecuencia

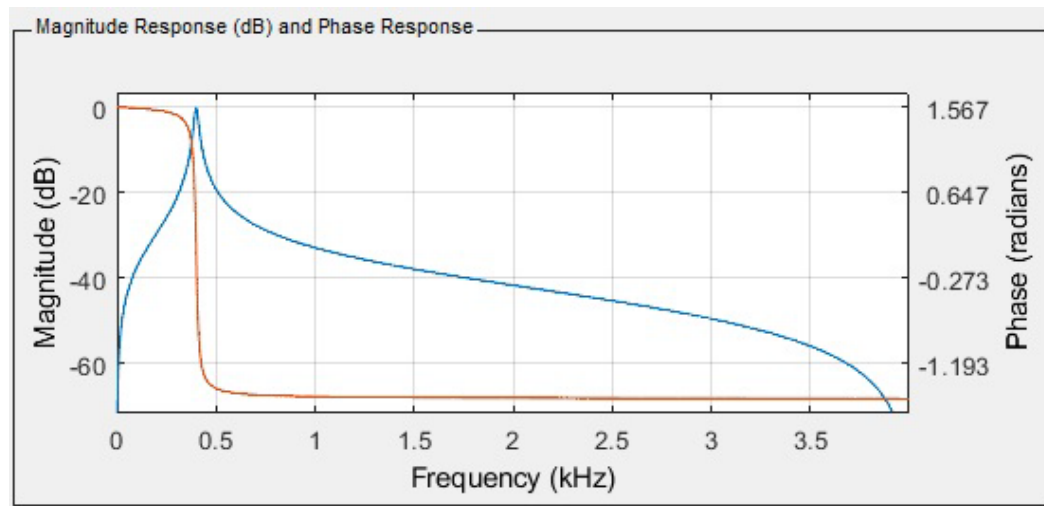


Patrón de Polos y Ceros

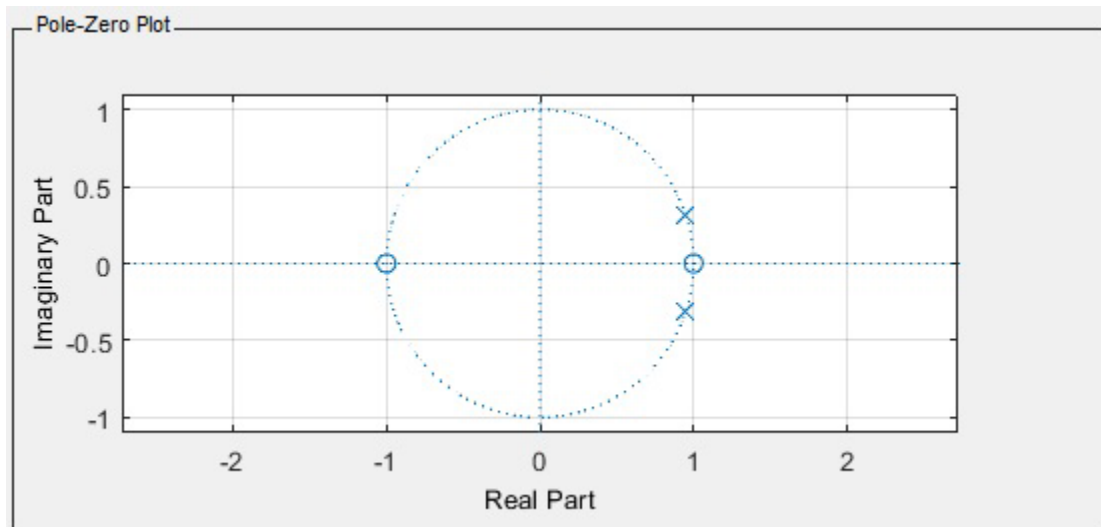


Coeficientes del Filtro

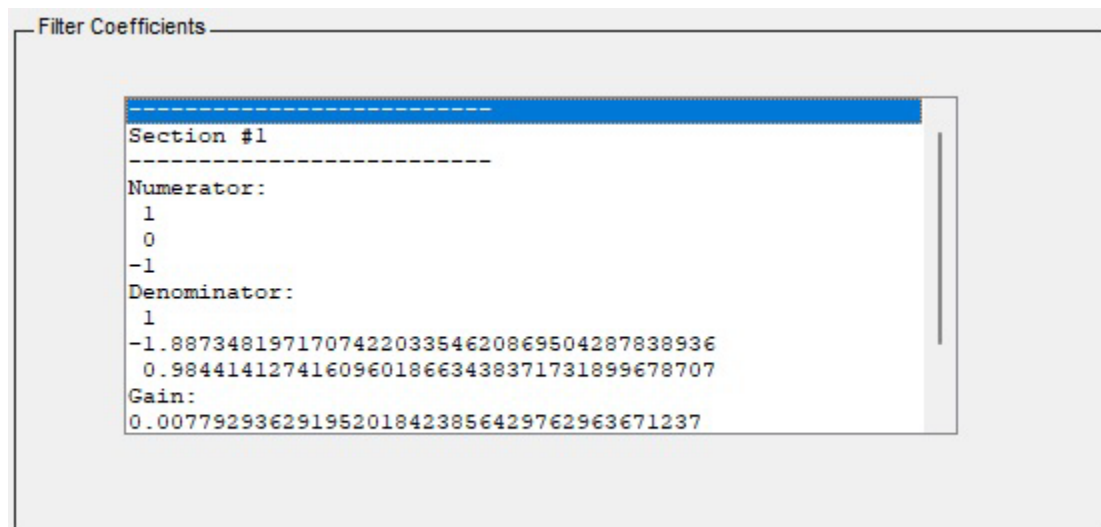
9. Pulsando el icono de “*Graficas de la Fase y Respuesta en Frecuencia*” se puede observar:



10. Con el icono del “*Patrón de Polos y Ceros*” se tiene:



11. Con el icono de relativo a “*Coeficientes del Filtro*” se tiene:



```

Filter Coefficients
-----
Numerator:
 1
 0
-1
Denominator:
 1
-1.887348197170742203354620869504287838936
 0.984414127416096018663438371731899678707
Gain:
0.007792936291952018423856429762963671237
-----
Output Gain:
1

```

12. Para formar la “Función de Transferencia” en base a la información anterior se tiene:

$$H(z) = \left[\frac{1 * Z^2 + 0 * Z - 1}{Z^2 - 1.8873481971Z + 0.9844141274} \right] * (Gain) * (Output Gain)$$

Dando los valores de “Gain” y “Output Gain”:

$$H(z) = \left[\frac{Z^2 - 1}{Z^2 - 1.8873481971Z + 0.9844141274} \right] * (0.0077929362) * (1)$$

La Función de Transferencia Discreta es:

$$H(z) = \frac{0.0077929362Z^2 - 0.0077929362}{Z^2 - 1.8873481971Z + 0.9844141274}$$

Desnormalizando la H(z) que equivale a dividir entre Z^2 se tiene:

$$H(z) = \frac{0.0077929362 - 0.0077929362Z^{-2}}{1 - 1.8873481971Z^{-1} + 0.9844141274Z^{-2}} = \frac{Y(z)}{U(z)}$$

Descomponiendo en términos de salidas y entradas:

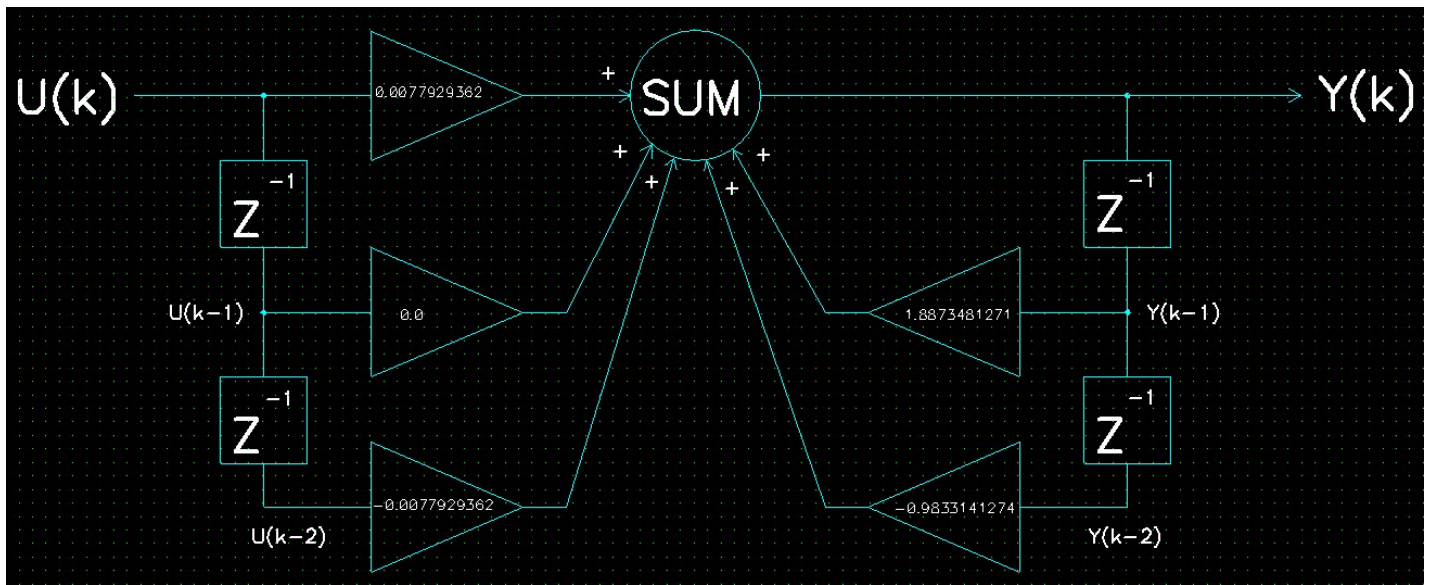
$$Y(z)[1 - 1.8873481971Z^{-1} + 0.9844141274Z^{-2}] = U(z)[0.0077929362 - 0.0077929362Z^{-2}]$$

Antitransformando:

$$Y(k) - 1.8873481971Y(k-1) + 0.9833141274Y(k-2) = 0.0077929362U(k) - 0.0077929362U(k-2)$$

La ecuación en diferencias resultante es:

$$Y(k) = 0.0077929362U(k) - 0.0077929362U(k-2) + 1.8873481971Y(k-1) - 0.9833141274U(k-2)$$



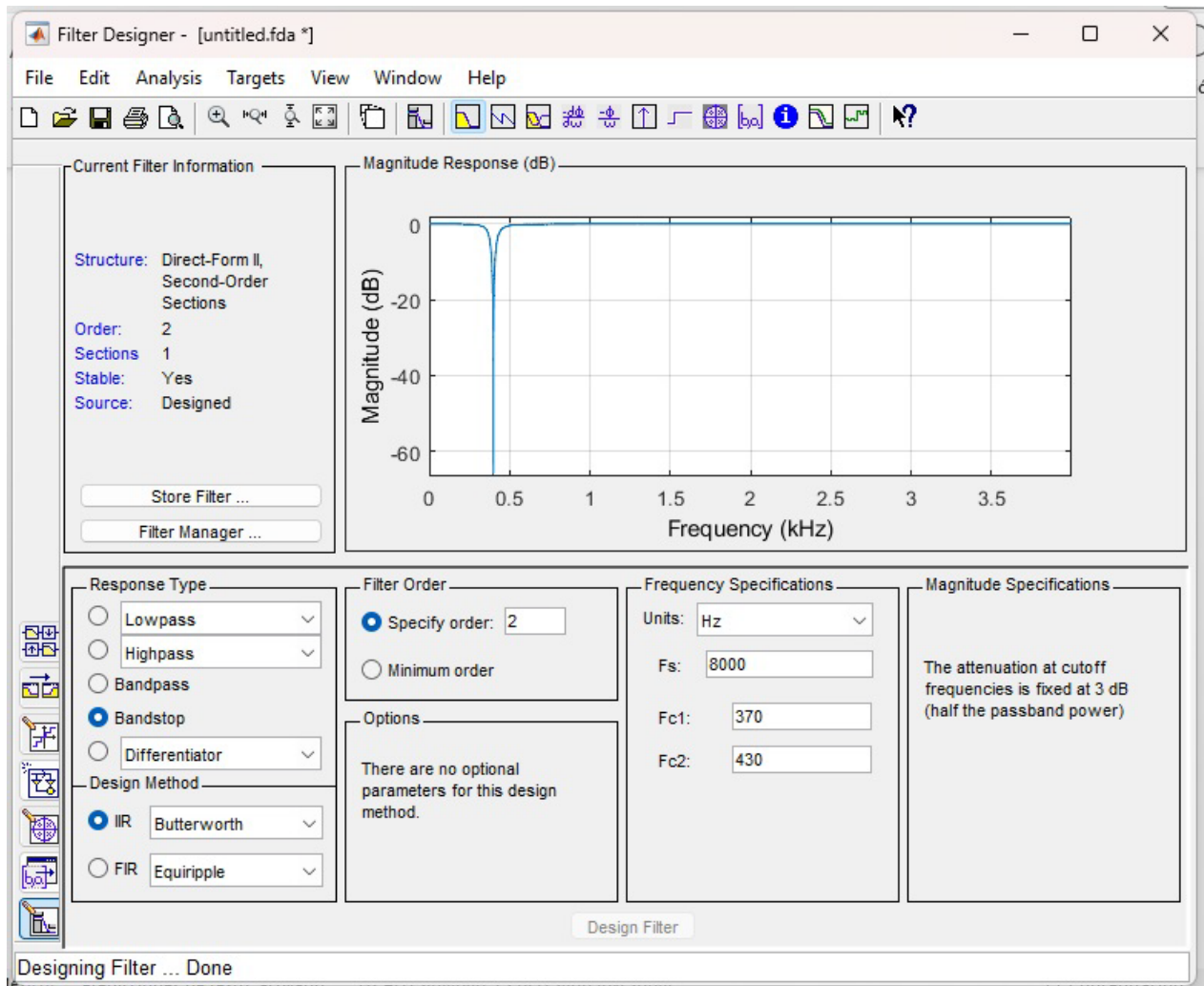
Aunque más adelante se discutirá la estructura de programa en Python implementado en la Raspberry Pi Pico y el software desarrollado en el “*appdesigner*” de MATLAB, al probar el filtro pasa banda se obtuvo:



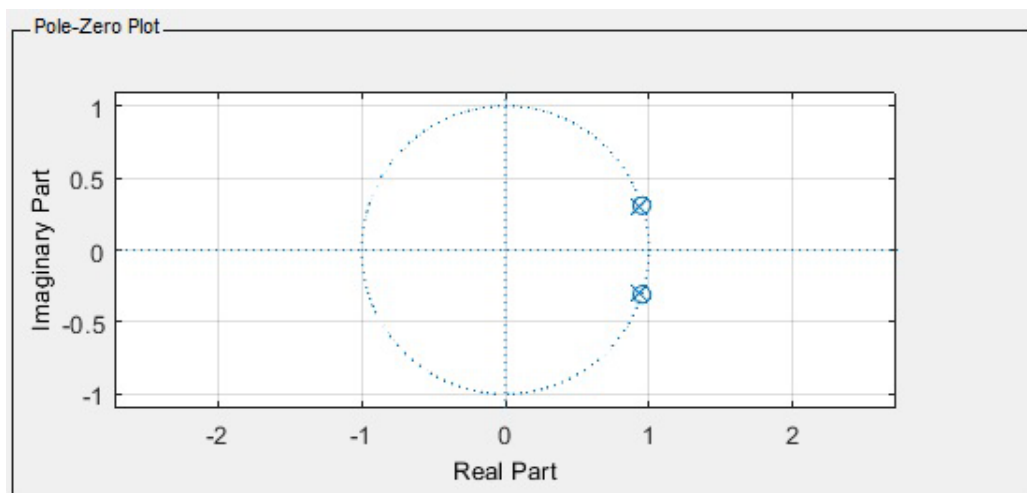
Cabe hacer notar que al tratarse de un filtro pasa banda alimentado mediante una onda cuadrada, este sólo deja pasar la primera armónica, por lo que, puede verse que en la salida del filtro se obtiene una onda senoidal (en color rojo) que coincide con el trazo de la gráfica de respuesta en frecuencia. En cambio, como la entrada es una onda cuadrada (en verde), su gráfica de respuesta en frecuencia presenta las armónicas características de una onda cuadrada, es decir, primera armónica, tercera armónica, quinta armónica, séptima armónica, novena armónica. Además, como frecuencia de la señal de entrada coincide con la frecuencia central del filtro ($f_0 = 400\text{Hz}$), se puede observar que la señal de salida está en fase con la señal de entrada.

EJEMPLO 5. Implementación de un filtro supresor de banda Butterworth de segundo orden en donde la frecuencia a suprimir es $f_o = 400\text{Hz}$ y un ancho de banda $BW = f_{c2} - f_{c1} = 430 - 370 = 60\text{Hz}$

Para este proceso de diseño también se utilizará la herramienta “*fdatool*” de MATLAB (vea el archivo *StopBand_400Hz/STOP_BAND_400Hz.py*).



El patrón de polos y ceros correspondiente al diseño por plantilla es:



Los coeficientes del filtro para formar la función de transferencia $H(z)$ son:

```

Filter Coefficients
-----
Numerator:
1
-1.902641148279058169734412331308703869581
1
Denominator:
1
-1.858835267327573648188376864709425717592
0.953952555907763288800538248324301093817
Gain:
0.976976277953881644400269124162150546908
-----
Output Gain:
1

```

Por lo tanto, la función de transferencia resultante es:

$$H(z) = \left[\frac{Z^2 - 1.902641148279Z + 1}{Z^2 - 1.85883526732757Z + 0.95395255907763288} \right] * (Gain) * (Output Gain)$$

$$H(z) = \left[\frac{Z^2 - 1.902641148279Z + 1}{Z^2 - 1.85883526732757Z + 0.95395255907763288} \right] * (0.9769762779538816444) * (1)$$

Finalmente, la función de transferencia $H(z)$ es:

$$H(z) = \left[\frac{0.9769762779538816444Z^2 - 1.858835267327573Z + 0.9769762779538816444}{Z^2 - 1.858835267327573648188Z + 0.9539525559077632888} \right] = \frac{Y(z)}{U(z)}$$

Dividiendo entre Z^2 se tiene entonces:

$$H(z) = \left[\frac{0.9769762779538816444Z^0 - 1.858835267327573Z^{-1} + 0.9769762779538816444Z^{-2}}{Z^0 - 1.858835267327573648188Z^{-1} + 0.9539525559077632888Z^{-2}} \right] = \frac{Y(z)}{U(z)}$$

Descomponiendo en términos de salidas y entradas:

$$Y(z)[1 - 1.858835267327573648188Z^{-1} + 0.9539525559077632888Z^{-2}] = U(z)[0.9769762779538816444 - 1.858835267327573Z^{-1} + 0.9769762779538816444Z^{-2}]$$

Antitransformando se obtiene:

$$Y(k) - 1.858835267327573648188Y(k-1) + 0.9539525559077632888Y(k-2) = 0.9769762779538816444U(k) - 1.858835267327573U(k-1) + 0.9769762779538816444U(k-2)$$

Despejando la salida actual $Y(k)$:

$$Y(k) = 0.9769762779538816444U(k) - 1.858835267327573U(k-1) + 0.9769762779538816444U(k-2) + 1.858835267327573648188Y(k-1) - 0.9539525559077632888Y(k-2)$$

En el programa de la Raspberry Pi Pico se tendría que modificar:

```

CTE_A0 = +0.976976277953388
CTE_A1 = -1.858835267327573
CTE_A2 = +0.976976277953388
CTE_B1 = +1.858835267327573
CTE_B2 = -0.9539525559077632888

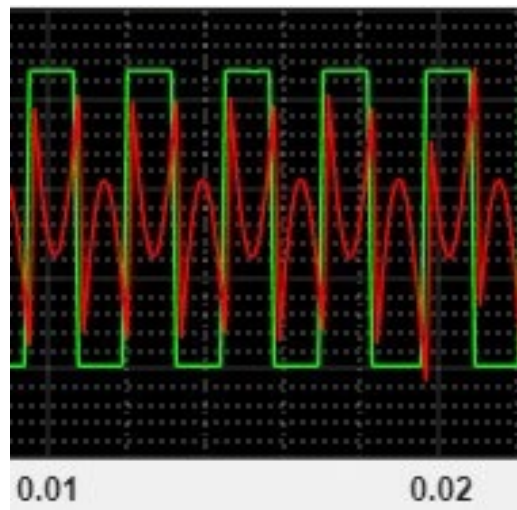
```

Gráfica de respuesta del filtro Buterworth supresor de banda con $f_o = 400 \text{ Hz}$.



NOTA 1: en la gráfica de respuesta en frecuencia se está suprimiendo los 400Hz (*la primera armónica*). La gráfica en el dominio del tiempo sólo se está presentando una señal que equivale a la sumatoria del 3ero, 5to, 7mo y 9no armónicos (*esto se puede corroborar viendo su gráfica en el dominio de la frecuencia*).

Detalle de la gráfica correspondiente a la señal de salida en el dominio del tiempo:

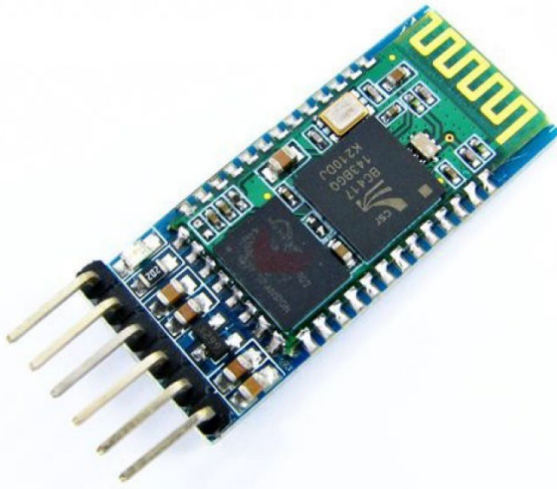


NOTA 2: el espectro de armónicos de la señal de entrada (*en verde*), presenta los armónicos 1ero, 3ero, 5to, 7mo y 9no. Aparentemente no se ven color verde debido a que se encuentra en traslape con la gráfica de la señal de salida.

APÉNDICE A

CÓMO CONFIGURAR EL MÓDULO BLUETOOTH HC-05

M. en I. José Antonio de Jesús Arredondo Garza



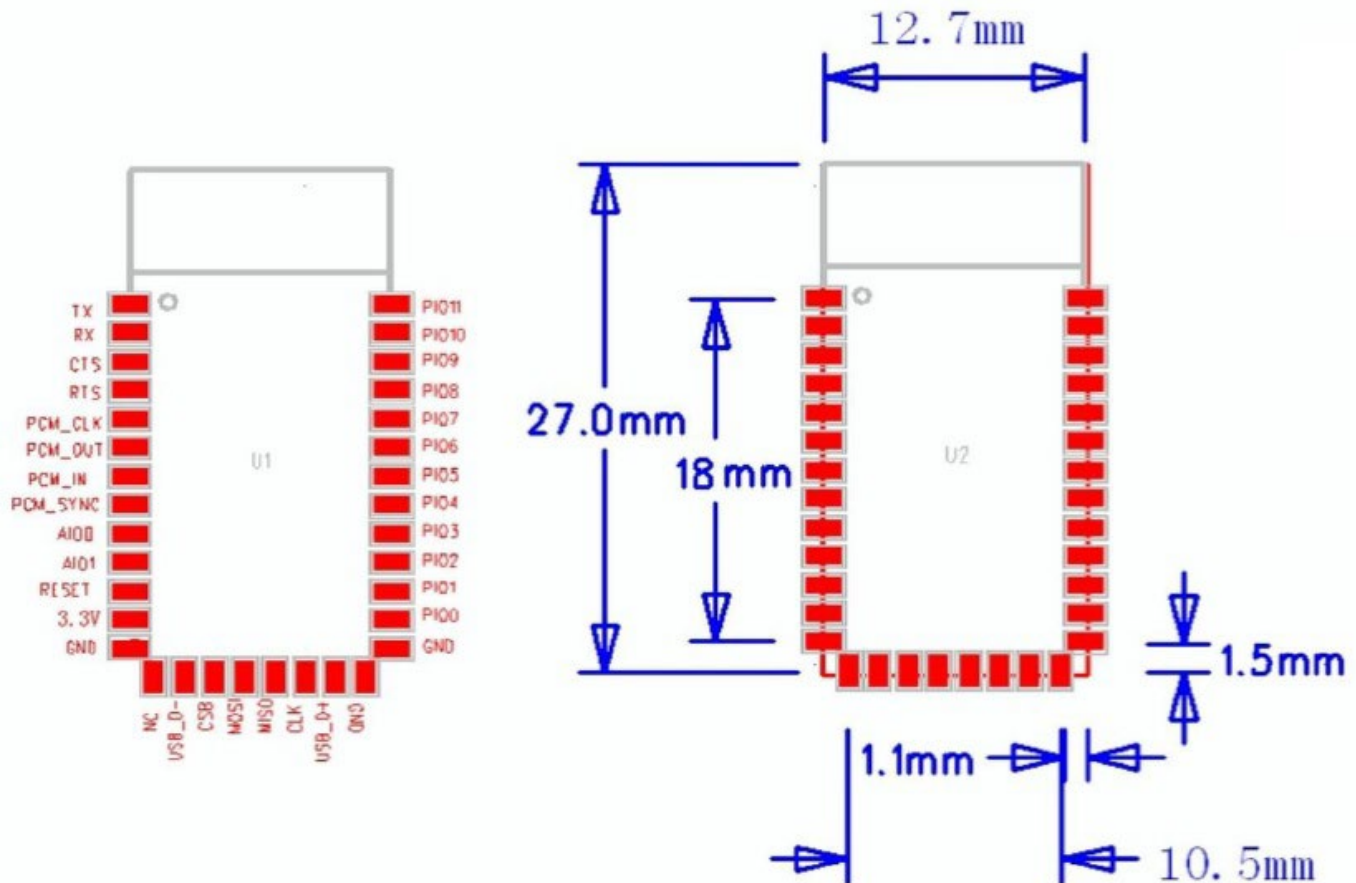
El módulo HC-05 tiene dos modos de operación (maestro o esclavo). Sin embargo, el módulo viene configurado de fábrica como esclavo.

FUNCIONES DE SOFTWARE (del datasheet)

- Velocidad de transmisión predeterminada: 38400, bits de datos: 8, bit de start: 1, bit de stop: 1, paridad: sin paridad, control de datos: Tasa de baudios admitida: 9600,19200,38400,57600,115200,230400,460800.
- Dando un pulso ascendente en el pin PIO0, el dispositivo se desconectará (en el módulo físico no se tiene este pin directamente disponible, por lo que, deberá pulsarse directamente a los pines soldados de la tarjeta).
- Contiene un Puerto de Estado de Instrucción PIO1: bajo desconectado, alto conectado.
- PIO10 y PIO11 se pueden conectar al LED rojo y azul por separado. Cuando el maestro y el esclavo están emparejados, el LED rojo y azul parpadea 1 vez/2 s en intervalos, mientras que cuando está desconectado, solo el LED azul parpadea 2 veces/s.
- Conexión automática al último dispositivo encendido de forma predeterminada.
- Permite que el dispositivo de emparejamiento se conecte de forma predeterminada.
- **CÓDIGO PIN** de emparejamiento automático: "0000" por defecto.
- Reconexión automática en 30 minutos cuando se desconecta como resultado de estar fuera del rango de conexión.

NOTA IMPORTANTE: para pruebas iniciales, puentee las terminales **RX** y **TX** del módulo HC-05 y use la **"Terminal Bluetooth de Windows"** para ver si se genera un eco con el módulo bluetooth.

EL HARDWARE (*del datasheet*)



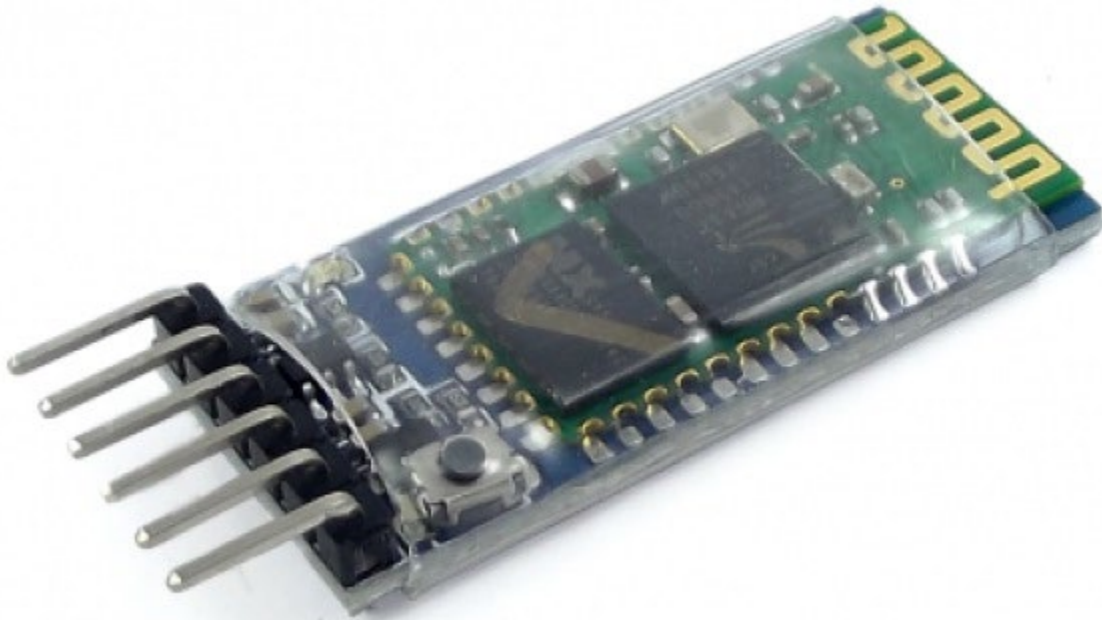
PIN Name	PIN #	Pad type	Description	Note
GND	13 21 22	VSS	Ground pot	
3.3 VCC	12	3.3V	Integrated 3.3V (+) supply with On-chip linear regulator output within 3.15-3.3V	
AIO0	9	Bi-Directional	Programmable input/output line	
AIO1	10	Bi-Directional	Programmable input/output line	
PIO0	23	Bi-Directional RX EN	Programmable input/output line, control output for LNA(if fitted)	
PIO1	24	Bi-Directional TX EN	Programmable input/output line, control output for PA(if fitted)	

PIO2	25	Bi-Directional	Programmable input/output line	
PIO3	26	Bi-Directional	Programmable input/output line	
PIO4	27	Bi-Directional	Programmable input/output line	
PIO5	28	Bi-Directional	Programmable input/output line	
PIO6	29	Bi-Directional	Programmable input/output line	
PIO7	30	Bi-Directional	Programmable input/output line	
PIO8	31	Bi-Directional	Programmable input/output line	
PIO9	32	Bi-Directional	Programmable input/output line	
PIO10	33	Bi-Directional	Programmable input/output line	
PIO11	34	Bi-Directional	Programmable input/output line	

RESETB	11	CMOS input with weak internal pull-up	Reset if low.input debounced so must be low for >5MS to cause a reset	
UART_RTS	4	CMOS output, tri-stable with weak internal pull-up	UART request to send, active low	
UART_CTS	3	CMOS input with weak internal pull-down	UART clear to send, active low	
UART_RX	2	CMOS input with weak internal pull-down	UART Data input	
UART_TX	1	CMOS output, Tri-stable with weak internal pull-up	UART Data output	
SPI_MOSI	17	CMOS input with weak internal pull-down	Serial peripheral interface data input	

USB_+	20	Bi-Directional		
NC	14			
PCM_CLK	5	Bi-Directional	Synchronous PCM data clock	
PCM_OUT	6	CMOS output	Synchronous PCM data output	
PCM_IN	7	CMOS Input	Synchronous PCM data input	
PCM_SYNC	8	Bi-Directional	Synchronous PCM data strobe	

Existe una versión del módulo **HC-05** que contiene un botón de configuración y la otra (que es la más común) sin este botón. Lo que hace este botón es conectar el **pin EN** al potencial de **VCC**. Este botón es usado para configurar a la tarjeta. En su ausencia se tiene que conectar manualmente el **pin EN** a **VCC**. Más adelante se explicará cómo se realiza este proceso.



DEFINICIONES SOBRE LAS CONFIGURACIONES MAESTRO Y ESCLAVO DEL H-C05

ESCLAVO. En esta configuración el módulo HC-05 se comporta de manera similar a un módulo HC-06 y espera que algún dispositivo bluetooth se conecte a este. Es ideal para comunicarse con dispositivos tales que una computadora personal, un celular u otro dispositivo. En este caso se asume que se estos dispositivos se están comportando como maestros.

MASTER. El módulo H-C05 en este modo es el que inicial la conexión. Este modo de operación se utiliza para comunicarse entre otros módulos bluetooth y es necesario especificar con que dispositivo se tiene que comunicar.

Cabe aclarar que el módulo comercial que se adquiere viene con 6 pines y a diferencia de la información presentada (proveniente del datasheet), la configuración inicial tiene las siguientes características:

- Modo inicial: SLAVE
- Nombre por defecto: HC-05
- Código de emparejamiento por defecto: 1234
- Baudaje inicial: 9600 bauds.

Asimismo, el módulo HC-05 tiene 4 modos de operación que a continuación se van a explicar:

1. ESTADO DESCONECTADO.

- Cuando se polariza el módulo este es el estado inicial y se asumen que no está conectado a ningún dispositivo.
- En este estado el LED del módulo parpadea rápidamente.
- En este estado, el HC-05 no puede interpretar ningún comando AT.

2. ESTADO DE CONEXIÓN O DE COMUNICACIÓN.

- Cuando se establece una conexión el módulo HC-05 entra en este estado.
- El LED realiza un doble parpadeo.
- Los datos que ingresan al Pin RX son transmitidos por el módulo bluetooth y los datos que se reciben se devuelven por el Pin TX. Todo el proceso de comunicación es transparente.

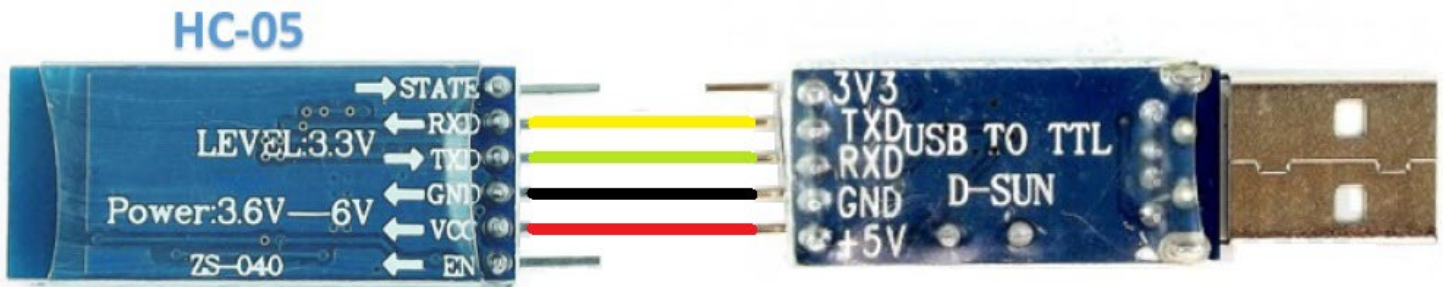
3. EL MODO “AT 1”

- Para entrar en este modo, después de encender el módulo, se deberá mantener pulsar el botón del **HC-05**. Si no tiene este botón, se deberá pulsar el **Pin EN** al potencial **VCC**.
- En este caso, se pueden enviar comandos AT al mismo baudaje con el que esté configurado.
- El LED estará parpadeando a la misma tasa que en el estado de desconexión.

4. EL MODO “AT 2”

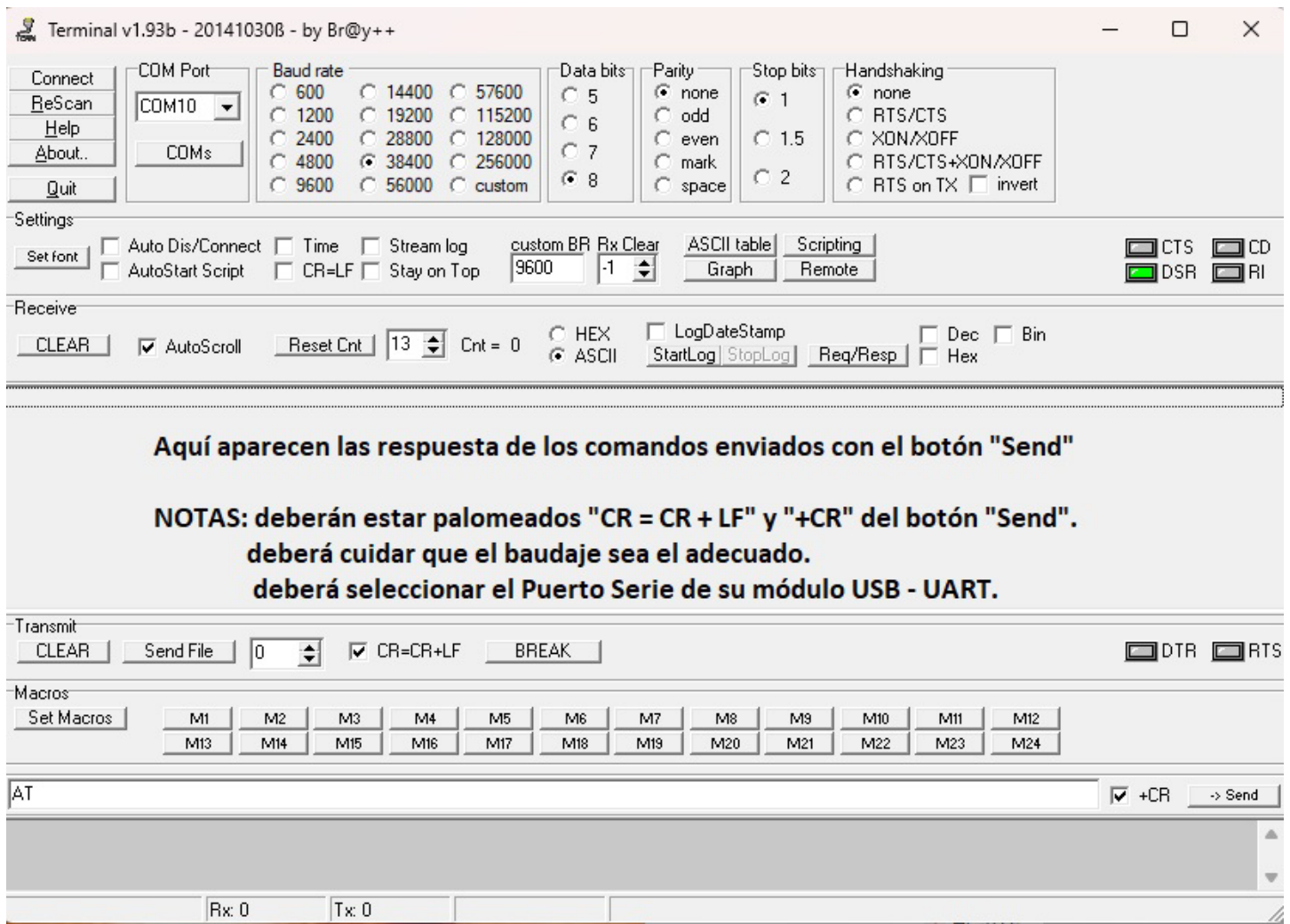
- En este caso, es necesario tener inicialmente presionado el botón (o el Pin EN a VCC). Después del encendido, se puede soltar el botón pues ya se entró en este modo de operación.
- En este estado se pueden enviar comandos AT, pero la tasa de baudios es de 38400 bauds. Este modo es ideal para cuando hemos olvidado a que tasa de baudios se configuró.
- El LED en este estado parpadea lentamente.

CONEXIONES INICIALES PAR LA CONFIGURACIÓN DEL MÓDULO HC-05



Existe en el mercado varios módulos **USB – UART**, y en principios todos funcionan de manera similar, por lo que, se puede elegir alguno que cumpla con las condiciones de la figura de arriba.

Para la programación se usó la terminal mostrada abajo:



NOTA IMPORTANTE: asigne exactamente los parámetros seleccionados en las casillas que están palomeadas y esté consciente si su módulos HC-05 está en el modo **“AT 1”** o en modo **“AT 2”** (seleccione el baudaje correspondiente al modo). Por supuesto, elija el puerto **COM** que su PC haya asignado a su módulo **USB – UART**.

LOS COMANDO “AT” BÁSICOS PARA UNA PROGRAMACIÓN RÁPIDA

1. Test command:

Command	Respond	Parameter
AT	OK	-

Set/Check module name:

Command	Respond	Parameter
AT+NAME=<Param>	OK	Param: Bluetooth module name (Default :HC-05)
AT+NAME?	+NAME:<Param> OK (/FAIL)	

Ejemplo:

ENVIAR: AT+NAME = RP2_BLUE *(con ese nombre va a aparecer al escanear con la PC)*

RESPUESTA: OK

Set/Check serial parameter:

Command	Respond	Parameter
AT+UART=<Param>,<Param2>,<Param3>	OK	Param1: Baud Param2: Stop bit Param3: Parity
AT+ UART?	+UART=<Param>,<Param2>,<Param3> OK	

Ejemplo:

ENVIAR: AT+UART = 460800,0,0 *(es el baudaje máximo en la UART de la Raspberry Pi Pico)*

RESPUESTA: OK

Nota: en este caso el baudaje será de 460800 bauds, 1 bit de stop, sin paridad.

Set/Check module mode:

Command	Respond	Parameter
AT+ROLE=<Param>	OK	Param: 0- Slave
AT+ ROLE?	+ROLE:<Param>	

Ejemplo:

ENVIAR: AT+ROLE = 0 *(modo esclavo)*

RESPUESTA: OK

Si desea ver la tabla de comandos completa consulte el datasheet del HC-05 y se recomienda ver los videos de youtube:

<https://youtu.be/sHgP15c8j1M>

<https://youtu.be/zoJUAjW3vQ0>

Ejemplo de programación en Micropython usando la tarjeta Raspberry Pi Pico y el módulo bluetooth HC-05

```

1 #####
2 #####XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX#####
3 #####
4 #####
5 ##### RUTINA EJEMPLO PARA MOSTRAR EL USO DE LA INTERFÁZ UART0 DE LA RASPBERRY #####
6 ##### PI PICO QUE TIENE EL NÚCLEO RP2040. #####
7 #####
8 ##### NOTA 1: SE TIENE CONECTADO UN MÓDULO BLUETOOTH TIPO "HC05" CONFIGURADO #####
9 ##### A UN BAUDAJE DE 460800 BAUDS, 1 BIT DE START, 1 BIT DE STOP. #####
10 ##### NOTA 2: EL PROGRAMA ESTÁ DISEÑADO PARA RESPONDER EN FORMA DE UN ECO DE #####
11 ##### LOS CARACTERES RECIBIDOS DE LA INTERFAZ BLUETOOTH DE WINDOWS. #####
12 ##### NOTA 3: ESTE PROGRAMA USA EL "PIN0" PARA "TX" Y EL "PIN1" PARA "RX". #####
13 #####
14 #####
15 #####XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX#####
16 #####
17 #####
18 ##### UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO #####
19 ##### DEPARTAMENTO DE CONTROL Y ROBÓTICA #####
20 ##### DEPARTAMENTO DE COMPUTACIÓN #####
21 #####
22 #####
23 #####XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX#####
24 #####
25 #####
26 ##### M. EN I. JOSÉ ANTONIO DE JESÚS ARREDONDO GARZA #####
27 ##### EMAIL: jarredon@unam.mx #####
28 ##### MAYO 2023 #####
29 #####
30 #####
31 #####XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX#####
32 #####
33
34
35
36 # XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX #
37 # Hacer que la CPU trabaje a 270 Mhz #
38 # XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX #
39
40 machine.freq(27000000)
41
42 #####
43 # Carga la librería de retardos #
44 #####
45
46 from time import sleep, sleep_ms, sleep_us
47
48 #####
49 # Carga la librería de la UART y las #
50 # definiciones de Pines. #
51 #####
52
53 from machine import UART, Pin
54
55 #####
56 # Define a la UART1 con 460800 bauds #
57 #####
58
59 uart0 = UART(0, baudrate=460800, tx=Pin(0), rx=Pin(1))
60
61 Pin25 = Pin(25, Pin.OUT) # Pin testigo de actividad
62 # (Led nativo).
63
64
65
66 #####
67 ## ##
68 ## Ciclo while para recibir un string de caracteres ##

```



```

69 ## y retransmitirlo de nuevo a la terminal bluetooth ##
70 ## de Windows. ##
71 ## ##
72 #####
73
74 while True:
75     DATO = bytes()          # Variable para recepción
76                             # de los "strings".
77
78     #-----#
79     # Investiga si se han recibido #
80     # algunos caracteres.          #
81     #-----#
82     if uart0.any() > 0:
83         Pin25.on()           # Enciende Pin 25 que
84                             # actúa como testigo
85                             # de recepción.
86         DATO = uart0.read()  # Lee la cadena de
87                             # caracteres.
88         uart0.write(DATO)    # Produce el eco
89                             # regresando los
90                             # caracteres.
91         print(DATO)          # Imprime en ventana de
92                             # comandos el "string"
93                             # (sólo como referencia).
94         Pin25.off()          # Apaga el Pin 25.
95
96

```

NOTA IMPORTANTE: si tiene dudas sobre la funcionalidad de su módulo bluetooth HC-05, lo que puede hacer es polarizar su módulo y puentear las terminales **RX** y **TX** para forzar a que el módulo nos de un eco de transmisión/recepción. Se sugiere usar la terminal bluetooth de Windows, aunque, previamente se deberá hacer el reconocimiento en su **PC** del **HC-05**.

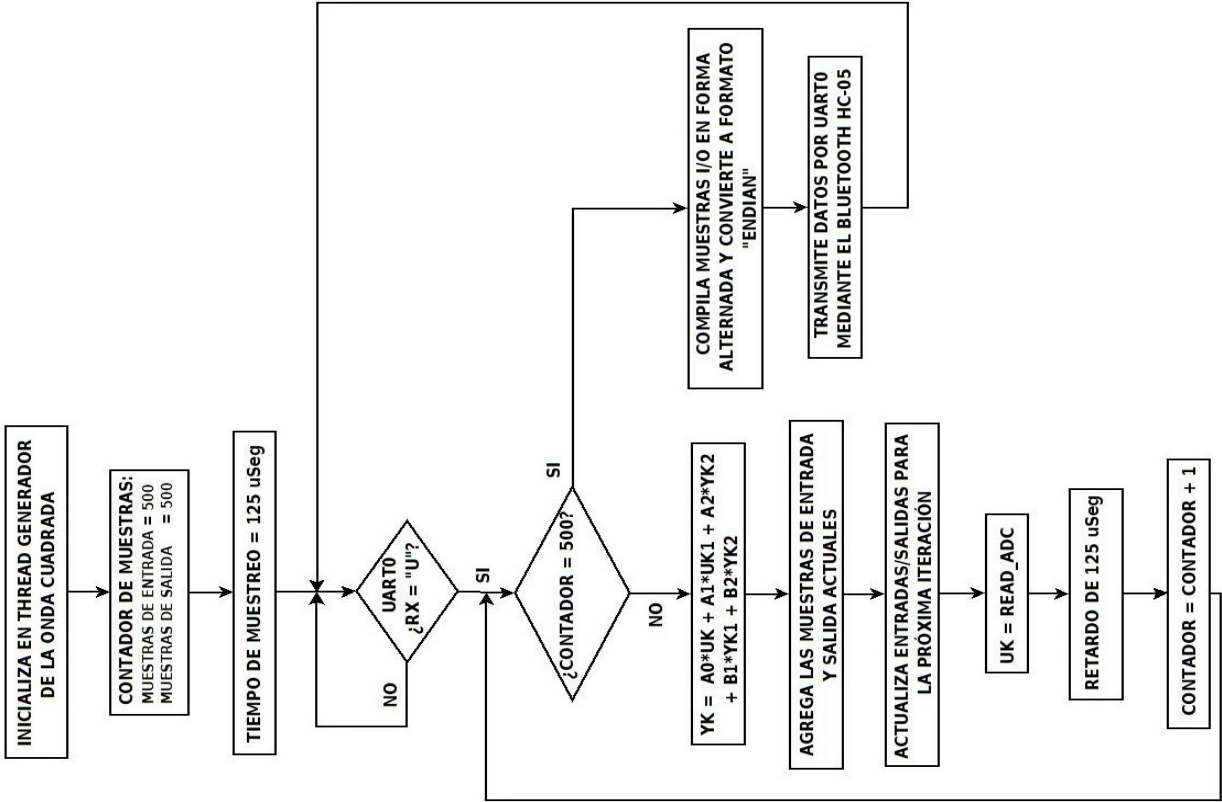
APÉNDICE B

El diagrama de flujo del programa en *Micropython* que hace el uso de hilos (*threads*) en la Raspberry Pi Pico es el siguiente:

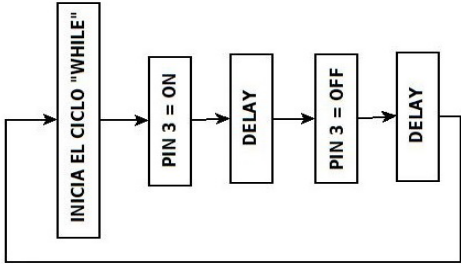
PROGRAMA PARA SIMULAR UN SISTEMA SUBAMORTIGUADO DISCRETO USANDO 2 NÚCLEOS

M. EN I. JOSÉ ANTONIO DE JESÚS ARREDONDO GARZA

RUTINA DE LA ECUACIÓN EN DIFERENCIAS



RUTINA DEL THREAD EN SEGUNDO PLANO



```

1 #####
2 #####
3 #####
4 ##### SIMULACIÓN DE UN CIRCUITO LRC BASADO EN LA RASPBERRY PI #####
5 ##### PICO UTILIZANDO LA INFRAESTRUCTURA DE SUS DOS NÚCLEOS. #####
6 #####
7 #####
8 #####
9 #####
10 ##### NOTA 1: EL PRIMER NÚCLEO ESTÁ DESTINADO A IMPLEMENTAR LA #####
11 ##### LA ECUACIÓN EN DIFERENCIAS. #####
12 ##### NOTA 2: EL SEGUNDO NÚCLEO GENERARÁ UNA ONDA CUADRADA. #####
13 ##### NOTA 3: LA FRECUENCIA DE MUESTREO ES DE 8 KHZ (125 uSEG), #####
14 ##### PERO DADO QUE EL PROGRAMA CONSUME TIEMPO DE #####
15 ##### EJECUCIÓN, EL RETARDO SE AJUSTÓ A 66 uSEG. #####
16 ##### LA SUMA DEL RETARDO DE EJECUCIÓN MÁS LOS 66 uSEG #####
17 ##### LA MUY APRÓXIMADAMENTE LOS 125 uSEG NECESARIOS. #####
18 ##### NOTA 4: LA ECUACIÓN EN DIFERENCIAS IMPLEMENTADA ES #####
19 #####  $Y(k) = A0*U(k) + A1*U(k-1) + A2*U(k-2)$  #####
20 #####  $+ B1*Y(k-1) + B2*Y(k-2)$  #####
21 ##### NOTA 5: SI SE DESEARA SIMULAR UNA ECUACIÓN EN DIFERENCIAS #####
22 ##### DE PRIMER ORDEN, SÓLO BASTA ASUMIR LOS COEFICIEN- #####
23 ##### "A2" Y "B2" IGUALES A CERO. #####
24 ##### NOTA 6: EN CADA "FRAME" SE ESTÁN TOMANDO 500 MUESTRAS DE #####
25 ##### ENTRADA Y 500 MUESTRAS DE SALIDA, LO CUAL ARROJA #####
26 ##### 1000 MUESTRAS EN CADA TRANSMISIÓN POR EL MÓDULO #####
27 ##### BLUETOOTH "HC-05". #####
28 ##### NOTA 7: SE ESTÁ USANDO LA UART0 DE LA RASPBERRY PI PICO A #####
29 ##### UNA TASA DE 460800 BAUDS. #####
30 ##### NOTA 8: LA COMPUTADORA PERSONAL DEBERÁ TRANSMITIR UN #####
31 ##### CARACTER "U" (0x85) PARA QUE LA RP2 INICIE EL #####
32 ##### PROCESO DE TRANSMISIÓN DEL "FRAME" DE 1000 #####
33 ##### MUESTRAS. #####
34 ##### NOTA 9: COMO MATLAB PRESENTÓ UN RARO PROBLEMA DE COMPRE- #####
35 ##### SIÓN DE LAS SEÑALES, SE LOGRÓ CORREGIR ESTO #####
36 ##### ENVIANDO LAS MUESTRAS EN FORMA ALTERNADA, ES DECIR, #####
37 ##### UNA ENTRADA/UNA SALIDA Y ASI SUCESIVAMENTE. #####
38 #####
39 #####
40 #####
41 #####
42 ##### ----- C ----- #####
43 ##### + | + #####
44 ##### ##### #####
45 ##### L #####
46 ##### #####
47 ##### Vi(t) | Vo(t) #####
48 ##### #####
49 ##### R #####
50 ##### - - #####
51 ##### | #####
52 ##### ----- #####
53 #####
54 #####
55 #####
56 #####
57 ##### PRESENTACIÓN DEL PROCESOS TEÓRICO PARA LA IMPLEMENTACIÓN #####
58 ##### DEL SISTEMA SUBAMORTIGUADO DESDE SU ORIGEN ANALÓGICO #####
59 ##### HASTA SU TRANSFORMACIÓN EN MODELO DISCRETO Y LA OBTENCIÓN #####
60 ##### DE LA ECUACIÓN EN DIFERENCIAS (UTILIZANDO UNA FRECUENCIA #####
61 ##### DE MUESTREO DE 8 KHZ): #####
62 #####
63 ##### EL MODELO MATEMÁTICO DEL CIRCUITO RLC: #####
64 #####  $H(s) = [S^2 + (R/L)S]/[S^2 + (R/L)S + (1/LC)]$  #####
65 ##### #####
66 #####  $H(s) = [S^2 + 2XiWoS]/[S^2 + 2XiWoS + Wo^2]$  #####
67 ##### #####
68 #####  $H(s) = [S^2 + 2XiWoS]/[(S + a)^2 + W^2]$  #####

```

```

69 #####
70 ##### Los únicos datos son: #####
71 #####
72 ##### f = 500 Hz ==> W = 2*pi*f = 3.1416e+03 [rad/seg] #####
73 #####
74 ##### Mp = 70% #####
75 #####
76 ##### PROCESO DE OBTENCIÓN DE EL COEFICIENTE DE AMORTIGUAMIENTO: #####
77 #####
78 ##### Mp = {Exp[(-Xi*Pi)/sqr[1 - (Xi)^2]}*100 #####
79 #####
80 ##### [70/100] = Exp[(-Xi*Pi)/sqr[1 - (Xi)^2] #####
81 #####
82 ##### ln(0.7) = [(-Xi*Pi)/sqr[1 - (Xi)^2] #####
83 ##### -0.3566749439 = [(-Xi*Pi)/sqr[1 - (Xi)^2] #####
84 ##### (-0.3566749439)sqr[1 - (Xi)^2] = (-Xi*Pi) #####
85 ##### (0.3566749439)sqr[1 - (Xi)^2] = (Xi*Pi) #####
86 #####
87 ##### ELEVANDO AL CUADRADO AMBOS LADOS DE LA ECUACIÓN: #####
88 ##### (0.127217)[1 - (Xi)^2] = (Xi*Pi)^2 #####
89 ##### 0.127217 = (Xi*Pi)^2 + [(Xi)^2]*0.127217 #####
90 #####
91 ##### FACTORIZANDO EL TÉRMINO Xi DE LA ECUACIÓN ANTERIOR SE TIENE: #####
92 ##### 0.127217 = [Xi^2][Pi^2 + 0.127217] #####
93 ##### [Xi^2] = [0.127217]/[Pi^2 + 0.127217] #####
94 ##### Xi = sqr{[0.127217]/[Pi^2 + 0.127217]} #####
95 #####
96 ##### Por lo tanto, Xi = 0.1128084507 #####
97 #####
98 ##### Wo = [W]/{[1 - Xi^2]^(0.5)} #####
99 #####
100 ##### Wo = (3141.59265e)/{[1 - 0.0127257465]^0.5} #####
101 #####
102 ##### Wo = 3161.775042082 [rad/seg] #####
103 #####
104 ##### Wo = 3.1618e+03 [rad/seg] #####
105 #####
106 ##### H(s) = [S^2 + (R/L)S]/[S^2 + 2*Xi*Wo*S + Wo^2] #####
107 #####
108 ##### SUSTITUYENDO VALORES EN LA FUNCIÓN DE TRANSFERENCIA: #####
109 #####
110 ##### H(s) = [S^2 + 713.34988S]/[S^2 + 713.34988S + 9996821.41673] #####
111 #####
112 #####
113 #####
114 ##### EL MÉTODO DE TUSTIN: S = (2/T)*{[Z - 1]/[Z + 1]} #####
115 #####
116 ##### Obtención de H(z) utilizando comandos en línea con MATLAB: #####
117 #####
118 ##### >>sys = tf([1 713.34988 0],[1 713.34988 9996821.41673]) #####
119 ##### >>sysd = c2d(sys,0.000125,'tustin') #####
120 #####
121 ##### MATLAB NOS REGRESA: #####
122 #####
123 ##### Sysd = [0.964Z^2 - 1.846Z + 0.8817] #####
124 ##### ----- #####
125 ##### Z^2 - 1.774Z + 0.9177 #####
126 #####
127 ##### LA ECUACIÓN EN DIFERENCIAS ES: #####
128 #####
129 ##### Y(k) = +0.964U(k) - 1.846U(k-1) + 0.8817U(k-2) #####
130 ##### +1.774Y(k-1) - 0.9177Y(k-2) #####
131 #####
132 ##### Que también equivale a la función de un filtro IIR. #####
133 #####
134 ##### NOTA: IIR = Infinite Impulse Response. #####
135 #####
136 ##### #####
137 ##### #####

```



```

207 #####XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX#####
208 #####XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX#####
209 #####                                                                 #####
210 ##### EQUIVALENTE DEL PROGRAMA PRINCIPAL EN MICROPYTHON #####
211 #####                                                                 #####
212 #####XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX#####
213 #####OXOXOXOXOXOXOXOXOXOXOXOXOXOXOXOXOXOXOXOXOXOXOXOXO#####
214 #####XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX#####
215
216
217 COUNTER = 0 # Contador de muestras.
218 conversion_factor = 3.3 / (65535) # Factor de conversión para
219                                     # que el A/D esté entre
220                                     # valores de 0 a 3.3 Volts.
221 MUESTRAS = 1000 # Número de muestras.
222 ENTRADAS = [0]*MUESTRAS # Variable que contendrá la
223                           # lectura del A/D (CANAL 0).
224                           # NOTA: esta variable contendrá
225                           # el npumero de muestras
226                           # (que en este caso serán
227                           # 500).
228 SALIDAS = [0]*MUESTRAS # Variable que contendrá las
229                          # salidas de Vo(k)'s.
230                          # NOTA: esta variable contendrá
231                          # el npumero de muestras
232                          # (que en este caso serán
233                          # 500).
234 En_Bytes = bytearray(8000) # Declara la variable que
235                             # contendrá el número de
236                             # muestras.
237                             # NOTA: un número en notación
238                             # de punto flotante está
239                             # compuesto de 4 bytes
240                             # (32 bits).
241
242
243 #####XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX#####
244 #####                                                                 #####
245 ##### LA ECUACIÓN EN DIFERENCIAS: #####
246 ##### Y(k) = +0.964U(k) - +0.964U(k-1) + 0.8817U(k-2) #####
247 #####          +1.774Y(k-1) - 0.9177Y(k-2) #####
248 #####                                                                 #####
249 #####XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX#####
250
251 CTE_A0 = +0.964
252 CTE_A1 = -1.846
253 CTE_A2 = +0.881
254 CTE_B1 = +1.774
255 CTE_B2 = -0.9177
256
257 ##XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX##
258 ## Condiciones Iniciales de las Entradas y Salidas. ##
259 ##XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX##
260
261 UK      = 0.0           # u(k1)
262 UK1     = 0.0           # u(k-1)
263 UK2     = 0.0           # u(k-2)
264 YK      = 0.0           # y(k)
265 YK1     = 0.0           # y(k-1)
266 YK2     = 0.0           # y(k-2)
267 Read    = 0.0
268
269
270 #####
271 #####XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX#####
272 #####                                                                 #####
273 ##### DEFINE LA RUTINA PARA GENERAR UNA ONDA CON CUADRADA EN #####
274 ##### SEGUNDO NÚCLEO DE LA RASPERRY PI PICO. #####
275 ##### NOTA: en este caso la frecuencia esta dada como #####

```

```

276 #####          f = (1/20mSeg) = 50 Hz          #####
277 #####          #####
278 #####XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX#####
279 #####
280
281 def Onda_Cuadrada():
282     while True:
283         Pin3.on()
284         sleep_ms(15)
285         Pin3.off()
286         sleep_ms(15)
287
288
289
290 #####
291 #####XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX#####
292 #####          #####
293 #####          RUTINA PRINCIPAL QUE ES LA RESPONSABLE DE INICIAR          #####
294 #####          EL "THREAD" EN 2DO PLANO PARA GENERAR LA ONDA          #####
295 #####          CUADRADA Y LA IMPLEMENTACIÓN EN 1ER PLANO DE LA          #####
296 #####          ECUACIÓN EN DIFERENCIAS.          #####
297 #####          #####
298 #####          NOTA: EL 1ER PLANO DE ESTA RUTINA IMPLEMENTA A LA          #####
299 #####          ECUACIÓN EN DIFERENCIAS CON UNA FRECUENCIA          #####
300 #####          DE MUESTREO DE 8 KHZ.          #####
301 #####          #####
302 #####XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX#####
303 #####
304
305 _thread.start_new_thread(Onda_Cuadrada,()) # Inicia el "thread" de
306                                           # la onda cuadrada.
307
308 ##-----##
309 ## Aquí inicia la implementación de la ##
310 ## Ecuación en Diferencias.          ##
311 ##-----##
312 while True:
313     P25.off()          # Apaga el Led testigo.
314
315     #####
316     # ¿Hay algún byte pendiente? #
317     #####
318     if UART0.any() > 0:
319         LLAVE = UART0.read(1)          # Lee el caracter recibido.
320
321         #x x x x x x x x x x x x #
322         # Si se recibió una "U" #
323         # (o sea un 0x55), sigue #
324         #x x x x x x x x x x x x #
325
326         if "U" in LLAVE:
327             P25.on()          # Led Testigo de Actividad.
328
329             ##x x x x x x x x x x x x x x x x x x x x x x x x x x x x ##
330             ## Captura 500 muestras a una tasa de muestreo Fs = 8Khz ##
331             ## provenientes de la iteraciones sucesivas de Vo(k). ##
332             ##x x x x x x x x x x x x x x x x x x x x x x x x x x x x ##
333
334             for i in range(0,500):
335
336                 #* * * * * #
337                 # Armado de la ecuación en diferencias #
338                 #* * * * * #
339
340                 YK = CTE_A0*UK+CTE_A1*UK1+CTE_A2*UK2+CTE_B1*YK1+CTE_B2*YK2
341
342                 #x x x x x x x x x x x x x x x x x x x x x x x x #
343                 # Actualiza I/O para la próxima iteración #
344                 #x x x x x x x x x x x x x x x x x x x x x x x x #

```

```

345      UK2 = UK1          # U(k-1) ----> U(k-2)
346      UK1 = UK           # U(k)   ----> U(k-1)
347      YK2 = YK1          # Y(k-1) ----> U(k-2)
348      YK1 = YK           # Y(k)    ----> Y(k-1)
349
350      Read = ADC0.read_u16() * conversion_factor # Toma lectura del canal 0
351                                                    # del ADC en volts reales.
352      UK = Read                                         # Asigna el nuevo valor U(k)
353      ENTRADAS[i] = UK                               # Compila las entradas.
354      SALIDAS[i] = YK                                # Compila las salidas.
355      sleep_us(66)                                     # Retardo de 125 uSeg.
356
357
358      P25.off()                                       # Apaga Led Testigo
359
360
361      *** * * * * * * * * * * * * * * * * * * * * **
362      ## Compilación Inicial de Entradas y Salidas ##
363      *** * * * * * * * * * * * * * * * * * * * * **
364
365      En_Bytes = bytearray(struct.pack("f",ENTRADAS[0]))
366      En_Bytes = En_Bytes + bytearray(struct.pack("f",SALIDAS[0]))
367
368      for k in range(1,500):
369
370          #x x x x x x x x x x x x x x x x x#
371          # Agregado de muestras subsiguientes #
372          #x x x x x x x x x x x x x x x x x#
373
374          En_Bytes = En_Bytes + bytearray(struct.pack("f", ENTRADAS[k]))
375          En_Bytes = En_Bytes + bytearray(struct.pack("f", SALIDAS[k]))
376
377      *** * * * * * * * * * * * * * * * * * * * * **
378      ## Transmisión de las Entradas y Salidas ##
379      ## por la UART0.                          ##
380      *** * * * * * * * * * * * * * * * * * * * * **
381
382      UART0.write(En_Bytes)
383
384      #####XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX#####
385      ## NOTA MUY IMPORTANTE: Si no se hacía este intercalado ##
386      ## de entradas salidas, se observó que se presentaba un ##
387      ## corrimiento en frecuencia indeseable al desplegar las ##
388      #####XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX#####
389
390

```

```

345         UK2 = UK1          # U(k-1) ----> U(k-2)
346         UK1 = UK           # U(k)   ----> U(k-1)
347         YK2 = YK1          # Y(k-1) ----> U(k-2)
348         YK1 = YK           # Y(k)   ----> Y(k-1)
349
350         Read = ADC0.read_u16() * conversion_factor # Toma lectura del canal 0
351                                                     # del ADC en volts reales.
352         UK = Read                                           # Asigna el nuevo valor U(k).
353         ENTRADAS[i] = UK                                   # Compila las entradas.
354         SALIDAS[i] = YK                                    # Compila las salidas.
355         sleep_us(66)                                       # Retardo de 125 uSeg.
356
357
358     P25.off()                # Apaga Led Testigo
359
360
361     ### * * * * *
362     ## Compilación Inicial de Entradas y Salidas ##
363     ### * * * * *
364
365     En_Bytes = bytearray(struct.pack("f",ENTRADAS[0]))
366     En_Bytes = En_Bytes + bytearray(struct.pack("f",SALIDAS[0]))
367
368     for k in range(1,500):
369
370         #x x x x x x x x x x x x x x x x#
371         # Agregado de muestras subsecuentes #
372         #x x x x x x x x x x x x x x x x#
373
374         En_Bytes = En_Bytes + bytearray(struct.pack("f", ENTRADAS[k]))
375         En_Bytes = En_Bytes + bytearray(struct.pack("f", SALIDAS[k]))
376
377     ### * * * * *
378     ## Transmisión de las Entradas y Salidas ##
379     ## por la UART0.                          ##
380     ### * * * * *
381
382     UART0.write(En_Bytes)
383
384     #####
385     ## NOTA MUY IMPORTANTE: Si no se hacía este intercalado ##
386     ## de entradas salidas, se observó que se presentaba un ##
387     ## corrimiento en frecuencia indeseable al desplegar las ##
388     #####
389
390

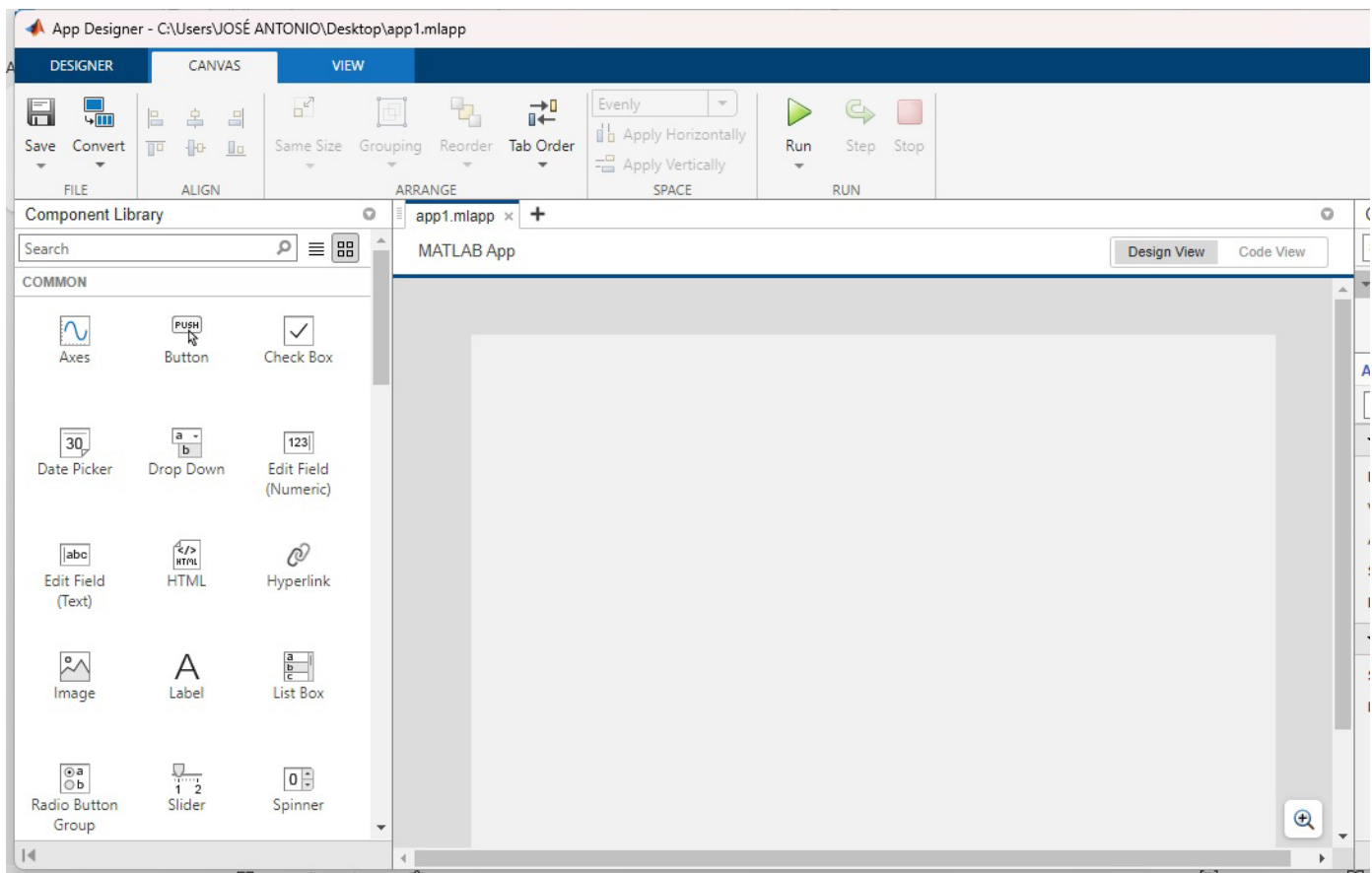
```

APÉNDICE C

CÓMO GENERAR UNA APLICACIÓN EN EL “appdesigner” DE MATLAB

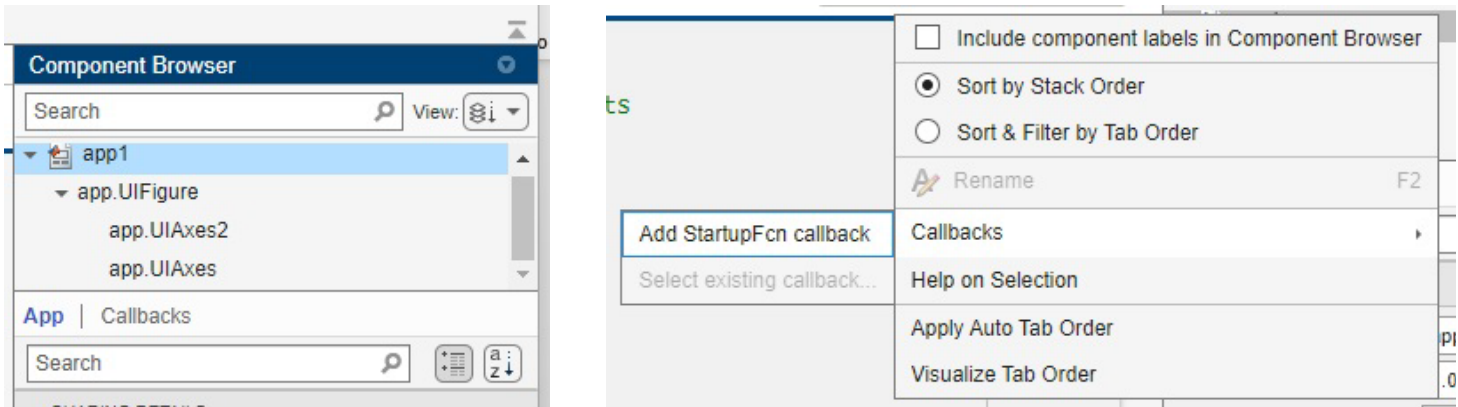
M. en I. José Antonio de Jesús Arredondo Garza

1. Inicie el paquete MATLAB.
2. En la línea de comandos teclee:
`>> appdesigner`
3. Inicie una aplicación en blanco (*blank*) y de una vez salve su aplicación con un nombre en una carpeta.
4. En este momento puede pulsar el botón verde de “Run” y arrancar la aplicación en vacío.



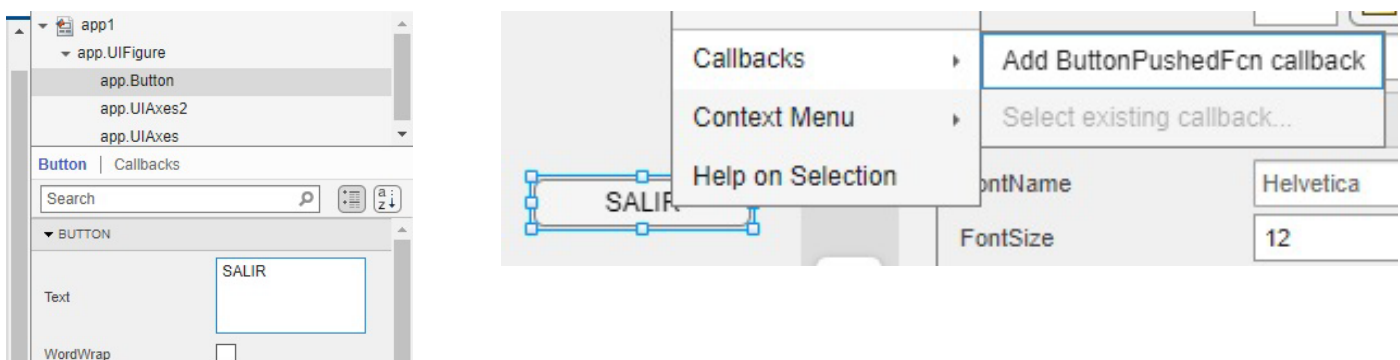
5. Aquí va a ver 2 botones “Design View” y “Code View”. El primero corresponde al modo de visualización gráfico de la aplicación y el segundo al modo de visualización del código de la aplicación. Va a notar que hay código escrito por default, el cual, no es modificable.
6. Va a visualizar la siguiente pantalla, en donde del lado derecho se encuentra el “Component Library” de donde vamos a traer los objetos que vamos a utilizar en esta aplicación. Además, observará que la aplicación en este momento se encuentra en blanco.
7. Vamos a arrastrar el ícono “axes” dos veces para formar 2 pantallas de graficación.

8. Va a hacer “click” con botón derecho del mouse sobre el nombre de la aplicación y luego en el desplegado va a hacer click sobre “add StartupFcn callback”, con lo cual, se va a generar un código sobre el cual vamos a programar las condiciones de inicio de la aplicación y por el momento sólo vamos a escribir “clear” para borrar cualquier interfaz externa que pudiera estar conectada.



```
% Code that executes after component creation
function startupFcn(app)
    clear;
    app.CONECTADOLamp.Color = "Red";
    app.GRAFICANDOLamp.Color = "Red";
    pause(0.25); % Retardo de 0.25 Seg
    warndlg(['¡ Asegúrese que el programa en la Raspberry Pi' ...
            ' Pico se esté ejecutando antes de pulsar el ' ...
            'botón CONECTAR !'], 'MENSAJE')
    app.CONNECT = 0; % Pon bandera de conexión en cero.
end
```

9. Ahora arrastrar un botón a la aplicación y le vamos a poner el nombre “SALIR”. Para poder darle nombre pulse del lado derecho donde aparece “app.Button”, asígnele el nombre y pulse el botón objeto con el botón derecho del mouse y pulse “Add ButtonPushFcn callback” para generar el código del botón.



10. Escriba el siguiente código para el botón “SALIR”, en donde se saldrá de la aplicación y se borrarán todas las interfaces abiertas.

```
% Button pushed function: SALIRButton
function SALIRButtonPushed(app, event)

%*****%
% Salir de la aplicación %
%*****%
close(app.UIFigure)
clear all;

end
```

11. Ahora se van a agregar los botones “CONECTAR” y “GRAFICAR” a y de igual manera que en el inciso 9 se agregarán los códigos correspondientes.



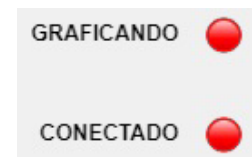
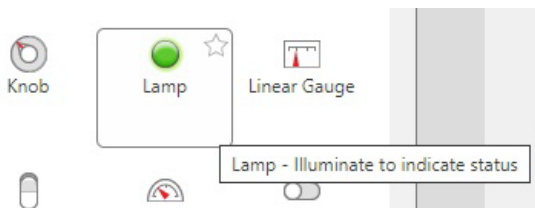
```
% Button pushed function: CONECTARButton
function CONECTARButtonPushed(app, event)

end

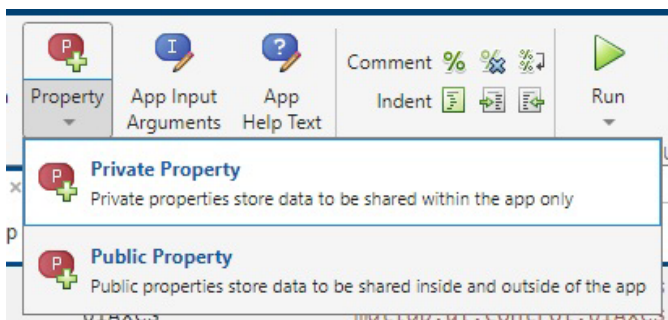
% Button pushed function: GRAFICARButton
function GRAFICARButtonPushed(app, event)

end
```

12. Se le van a integrar dos objetos “LED” con los nombres “CONECTADO” y “GRAFICAR”, los cuales deberán ponerse en color rojo inicialmente.



13. Se agregarán propiedades con carácter “private” que va a servir para declarar las variables que más adelante se usarán para la adquisición de datos y graficación

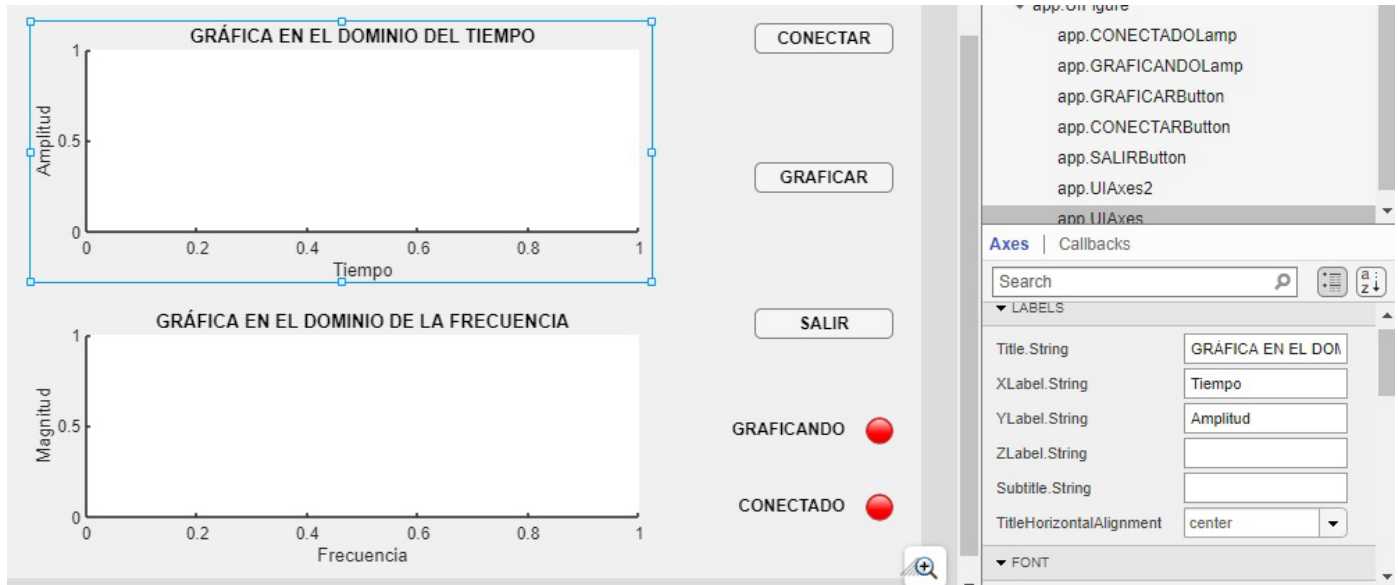


```
properties (Access = private)
%*****%
% DECLARACIÓN DE VARIABLES QUE SE USARÁN %
% EN ESTA APLICACIÓN. %
%*****%

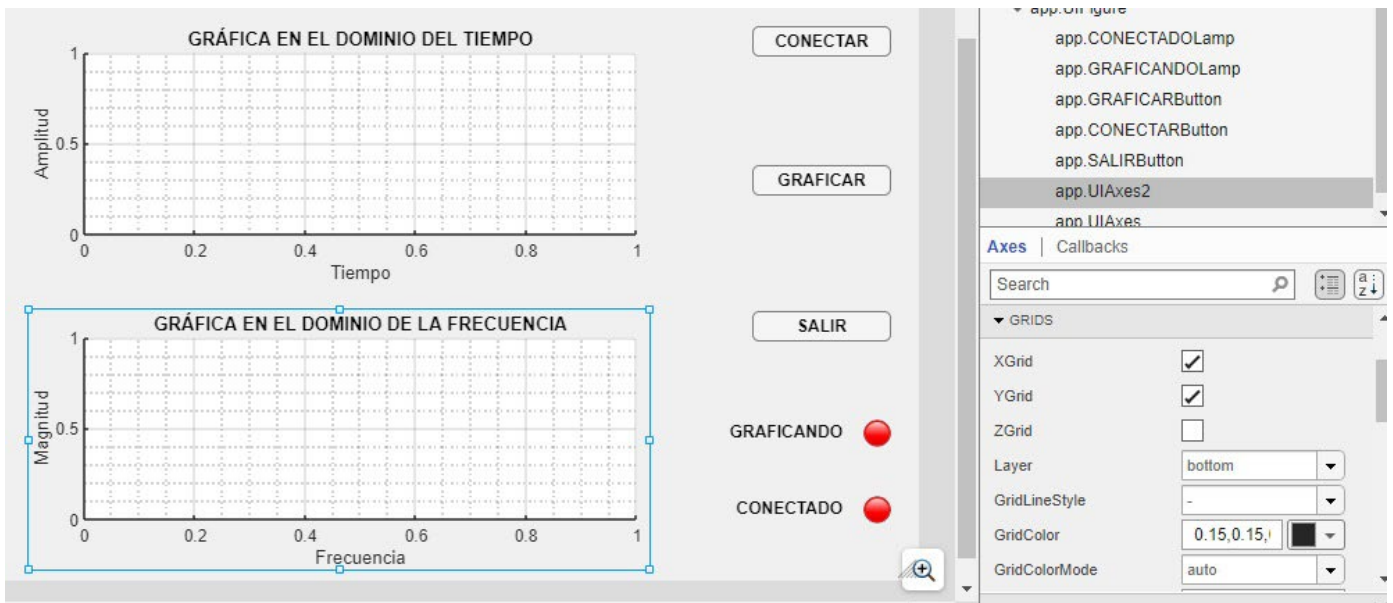
Size_nFFT = 1000; % Variable que indicará el número de
                  % muestras por canal y el tamaño de
                  % la FFT.
Fs = 1000; % Frecuencia de Muestreo.
Ts = 0.001; % Tiempo de Muestreo (Ts = 1/Fs),
t; % Variable usada para crear el eje del
    % tiempo en las gráficas.
f; % Variable usada para crear el eje de
    % la frecuencia.
CONNECT = 0; % Bandera para indicar conexión
PUERTO; % Variable que contendrá el objeto serial.

end
```

14. Ahora se dará nombre al título a las gráficas que presentará las señales en el dominio del tiempo y dominio de la frecuencia, y también se asignarán los nombres de los ejes.



15. Se crearán las líneas de las gráficas palomeando "XGrid", "YGrid", "XMinorGrid" e "YMinorGrid".



16. Para el código del botón “CONECTAR” se va a crear el “objeto serial” con el cual se realizará el enlace con la Raspberry Pi Pico. Nota: no olvide poner “end” al final de las sentencias “if” y “else”.

NOTA: si se usa el módulo bluetooth HC-05 se recomienda renombrar el módulo con el nombre “My_RP2” para tener una compatibilidad con *appdesigner* de MATLAB final.

```
% Button pushed function: CONECTARButton
function CONECTARButtonPushed(app, event)
    if app.CONNECT == 1
        warndlg(['; Ya está conectado !'], 'MENSAJE')
    else
        % ----- %
        % Define el puerto serie que se va a utilizar. %
        % NOTA: El buffer es 8000 porque son 2 canales porque %
        % cada canal aporta 1000 muestras en formato en formato %
        % "single" que se traducen en 2000*4 = 8000 bytes ENDIAN %
        % ----- %
        app.PUERTO = serialport(COM14,230400);
        app.PUERTO.InputBufferSize = 8000;
        fopen(app.PUERTO);

        % ----- %
        % Si se usa un módulo bluetooth se tendría como ejemplo: %
        % NOTA: aquí no se requiere asignar tamaño buffer, ni %
        % baudaje ya que eso depende de la configuración %
        % del módulo bluetooth (@ 230400 bauds). %
        % ----- %
        % app.PUERTO = bluetooth('RNBT-EE12', 1); % Donde RNBT-EE12
        % es el nombre
        % del dispositivo.

        % ----- %
        % Si se usa una conexión TCP CLIENT como ejemplo: %
        % NOTA: donde 65432 es el código de identificación entre %
        % el TCP CLIENT y el TCP SERVIDOR (puede ser una %
        % Raspberry Pi. %
        % ----- %
        % PUERTO = app.PUERTO = tcpclient('192.168.1.xxx', 65432,"Timeout",10)

        app.CONNECT = 1; % Bandera CONNECT=1
        app.CONECTADOLamp.Color = "Green"; % Led de conexión.

        % ----- %
        % Creación de los ejes de graficación %
        % ----- %

        % Eje del Tiempo
        app.t = (app.Size_nFFT-1)*app.Ts;
        % Eje de la Frecuencia
        app.f = (app.Size_nFFT/2 - 1)*(app.Fs/app.Size_nFFT);

        app.CONNECT = 1; % Pon bandera de conexión.
    end
end
```


17. Después de las declaraciones anteriores en el código del botón “GRAFICAR” se va a crear el “*loop de graficación*” (dentro del segmento “*else*”). En caso de no estar conectado se desplegará un letrero de “*no conexión*”. No olvide poner “end” al final de las sentencias “*if-else*”.

```
% Button pushed function: GRAFICARButton
function GRAFICARButtonPushed(app, event)
    if app.CONNECT == 0
        warndlg(['; No está conectado !'], 'MENSAJE')
    else
        for i = 1:50
            INPUT = zeros(1, 1000); % Pon la variable de muestras
                                   % de entrada en ceros.
            OUTPUT = zeros(1, 1000); % Pon la variable de muestras
                                   % de salida en ceros.
            app.GRAFICANDOLamp = "Green"; % Led de gradicación en
                                           % verde.

            % ----- %
            % Envía caracter "U" %
            % de sincronía.      %
            % ----- %
            fwrite(app.PUERTO,85,"uint8"); % Caso de una UART.
            % write(app.PUERTO,85,"uint8"); % Caso del bluetooth.
            % write(app.PUERTO,85,"uint8"); % Caso TCP.

            % ----- %
            % Lectura de Datos que son 1000 de INPUT y 1000 %
            % de OUTPUT (los cuales están alternados IN/OUT) %
            % ----- %
            % Caso UART
            READ = fread(app.PUERTO,app.Size_nFFT*2,"single");
            % Caso bluetooth
            % READ = read((app.PUERTO,app.Size_nFFT*2,"single");
            % Caso TCP CLIENT
            % READ = read((app.PUERTO,app.Size_nFFT*2,"single");

            % ----- %
            % Limpia el buffer de recepción (caso UART) %
            % NOTA: Hay que revisar si existe para los %
            % casos bluetooth y TCP.                  %
            % ----- %
            flush(app.PUERTO);
```

18. Se limpia el buffer “READ” y se separan las muestras de entrada/salida (continuación del “else”).

```
% ----- %
% Limpia el buffer de recepción %
% ----- %
flush(app.PUERTO);

% ----- %
% Separación de las muestras de entrada/salida %
% ----- %
r = 1;
for j = 1:1000
    INPUT(j) = READ(r);      % Muestras de entrada.
    OUTPUT(j) = READ(r + 1); % Muestras de salida.
    r = r + 1;               % Incrementa apuntador.
end
```

19. Obtención de las Transformadas Rápidas de Fourier (continuación del “else”).

```
% ----- %
% Obtención de las Transformadas Rápidas de Fourier %
% ----- %
IN_FFT = fft(INPUT,app.Size_nFFT); % FFT de IN.
IN_FFT = IN_FFT(1:app.Size_nFFT/2); % Decimación.
IN_FFTx = abs(IN_FFT);              % Valor absoluta

OUT_FFT = fft(OUTPUT,app.Size_nFFT); % FFT de OUT.
OUT_FFT = OUT_FFT(1:app.Size_nFFT/2); % Decimación.
OUT_FFTx = abs(OUT_FFT);            % Absoluto
```

20. Proceso de graficación (fin del “else”) y se pone el Led indicador de graficación en rojo.

```
% ----- %
% Proceso de Graficación en Tiempo y Frecuencia %
% ----- %

% Dominio del Tiempo.
plot(app.UIAxes,app.t,INPUT,'g',app.t,OUTPUT,'r');
drawnow

% Dominio de la Frecuencia.
plot(app.UIAxes2,app.f,IN_FFTx,'g',app.f,OUT_FFTx,'r');
drawnow

end

% ----- %
% Pon Led de Graficación en Rojo para indicar el %
% final de la graficación. %
% ----- %
app.GRAFICANDOLamp.Color = "Red";|

end
end
```


properties (Access = private)

```
%XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX%
%
%  UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO  %
%      FACULTAD DE INGENIERÍA                %
%  DEPARTAMENTO DE CONTROL Y ROBÓTICA         %
%
%      M. en I. José Antonio de Jesús        %
%      Arredondo Garza.                     %
%      Email: jarredon@unam.mx               %
%
%      MARZO 2022                           %
%
%XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX%
```

```
%XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX%
% DECLARACIÓN DE VARIABLES QUE SE %
% USARÁN EN ESTA APLICACIÓN.      %
%XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX%
```

```
Size_nFFT;          % Variable para contener el número de
                    % muestras (por canal).
Fs;                  % Variable que contendrá la Frecuencia
                    % de muestreo.
Ts;                  % Variable que contendrá el Tiempo de
                    % muestreo.
t;                   % Variable usada para crear el eje del
                    % tiempo en los ejes de graficación.
f;                   % Variable usada para crear el eje de
                    % la frecuencia en los ejes de
                    % graficación.
```

end

NOTA: Observe la declaración “*My_RP2*” para el módulo bluetooth HC-05

```
% Button pushed function: Conectar
function ConectarButtonPushed(app, event)
    %XXXXXXXXXXXXXXXXXXXXXXXXXXXXX%
    % CÓDIGO DEL BOTÓN CONECTAR %
    %XXXXXXXXXXXXXXXXXXXXXXXXXXXXX%

    %*****%
    % Asignación de valores a %
    % las variables. %
    %*****%
    app.Size_nFFT = 500; % Número de muestras
                        % (por canal).
    app.Fs = 8000; % Frecuencia de
                  % muestreo Fs = 8Khz
    app.Ts = (1/app.Fs); % Tiempo de muestreo.

    %*****%
    % Creación de los ejes del tiempo %
    % y de la frecuencia. %
    %*****%
    app.t = (0:app.Size_nFFT-1)*app.Ts;
    app.f = (0:app.Size_nFFT/2-1)*(app.Fs/app.Size_nFFT);

    %*****%
    % Creación del objeto bluetooth. %
    %*****%
    app.LED_Actividad.Color = 'Green'; % Pon en Verde al LED

    clear My_RP2; % Borra el puerto
                  % bluetooth por si
                  % acaso.
    PUERTO = bluetooth("My_RP2", 1); % Conectar bluetooth.

    %XOXOXOXOXOXOXOXOXOXOXOXOXOXOXOX%
    % Loop de adquisición y graficación %
    %XOXOXOXOXOXOXOXOXOXOXOXOXOXOXOX%

    i = 1;
    while i <= 20
        write(PUERTO, "U", "string"); % Transmite una "U"
                                      % (un 0x55), que es
                                      % el caracter guía
                                      % para iniciar el
                                      % proceso de recepción.

        %*****%
        % Recepción de muestras ya %
        % convertidas a single, que %
        % equivale al float de la %
        % Raspberry Pi Pico. %
        %*****%
        DATA = read(PUERTO, app.Size_nFFT*2, "single");
        flush(PUERTO) % Limpia los buffers.

        %*****%
        % Separación de los canales %
        % de datos. %
        %*****%
        N = DATA;

        Count = 1;
        k = 1;
        while Count <= 1000
            N1(k) = N(Count); % Muestras del Canal 1.
            N2(k) = N(Count + 1); % Muestras del Canal 2.
            Count = Count + 2; % Incrementa contador.
            k = k + 1; % Incrementa contador de

        end
    end
```



```

% Create UIAxes
app.UIAxes = uiaxes(app.UIFigure);
title(app.UIAxes, 'GRÁFICA EN EL DOMINIO DEL TIEMPO')
xlabel(app.UIAxes, 'Tiempo (Segundos)')
ylabel(app.UIAxes, 'Amplitud (Volts)')
app.UIAxes.FontWeight = 'bold';
app.UIAxes.Color = [0 0 0];
app.UIAxes.XGrid = 'on';
app.UIAxes.XMinorGrid = 'on';
app.UIAxes.YGrid = 'on';
app.UIAxes.YMinorGrid = 'on';
app.UIAxes.GridColor = [1 1 1];
app.UIAxes.MinorGridColor = [1 1 1];
app.UIAxes.Position = [93 285 1056 220];

% Create UIAxes2
app.UIAxes2 = uiaxes(app.UIFigure);
title(app.UIAxes2, 'GRÁFICA EN EL DOMINIO DE LA FRECUENCIA')
xlabel(app.UIAxes2, 'Frecuencia (Hertz)')
ylabel(app.UIAxes2, 'Magnitud')
app.UIAxes2.FontWeight = 'bold';
app.UIAxes2.Color = [0 0 0];
app.UIAxes2.XGrid = 'on';
app.UIAxes2.XMinorGrid = 'on';
app.UIAxes2.YGrid = 'on';
app.UIAxes2.YMinorGrid = 'on';
app.UIAxes2.GridColor = [1 1 1];
app.UIAxes2.MinorGridColor = [1 1 1];
app.UIAxes2.Position = [93 48 1056 220];

% Create Image
app.Image = uiimage(app.UIFigure);
app.Image.Position = [1 494 114 99];
app.Image.ImageSource = fullfile(pathToMLAPP, 'UNAM.jpg');

% Create Image2
app.Image2 = uiimage(app.UIFigure);
app.Image2.Position = [1148 494 114 99];
app.Image2.ImageSource = fullfile(pathToMLAPP, 'Ingenieria.jpg');

% Create OSCILOSCOPIODEDOSCANALESENTIEMPOYFRECUENCIABASADOLabel
app.OSCILOSCOPIODEDOSCANALESENTIEMPOYFRECUENCIABASADOLabel = uilabel(app.UIFigure);
app.OSCILOSCOPIODEDOSCANALESENTIEMPOYFRECUENCIABASADOLabel.HorizontalAlignment = 'center';
app.OSCILOSCOPIODEDOSCANALESENTIEMPOYFRECUENCIABASADOLabel.FontSize = 18;
app.OSCILOSCOPIODEDOSCANALESENTIEMPOYFRECUENCIABASADOLabel.FontWeight = 'bold';
app.OSCILOSCOPIODEDOSCANALESENTIEMPOYFRECUENCIABASADOLabel.Position = [202 570 925 23];
app.OSCILOSCOPIODEDOSCANALESENTIEMPOYFRECUENCIABASADOLabel.Text = 'OSCILOSCOPIO DE DOS CANALES EN TIEMPO Y FRECUENCIA';

% Create D
app.D = uilabel(app.UIFigure);
app.D.HorizontalAlignment = 'center';
app.D.FontSize = 14;
app.D.FontAngle = 'italic';
app.D.Position = [474 504 311 22];
app.D.Text = 'M. en I. José Antonio de Jesús Arredondo Garza';

```

```

% Create UNIVERSIDADNACIONALAUTNOMADEMEXICOFACULTADDEINGENIERALabel
app.UNIVERSIDADNACIONALAUTNOMADEMEXICOFACULTADDEINGENIERALabel = uilabel(app.UIFigure);
app.UNIVERSIDADNACIONALAUTNOMADEMEXICOFACULTADDEINGENIERALabel.FontSize = 14;
app.UNIVERSIDADNACIONALAUTNOMADEMEXICOFACULTADDEINGENIERALabel.FontAngle = 'italic';
app.UNIVERSIDADNACIONALAUTNOMADEMEXICOFACULTADDEINGENIERALabel.Position = [370 525 553 22];
app.UNIVERSIDADNACIONALAUTNOMADEMEXICOFACULTADDEINGENIERALabel.Text = 'UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO - FA

% Create Label
app.Label = uilabel(app.UIFigure);
app.Label.FontSize = 50;
app.Label.FontColor = [0 1 0];
app.Label.Position = [128 275 33 65];
app.Label.Text = '_';

% Create Label_2
app.Label_2 = uilabel(app.UIFigure);
app.Label_2.FontSize = 50;
app.Label_2.FontColor = [1 0 0];
app.Label_2.Position = [245 275 33 65];
app.Label_2.Text = '_';

% Create Canal1Label
app.Canal1Label = uilabel(app.UIFigure);
app.Canal1Label.Position = [168 270 47 22];
app.Canal1Label.Text = 'Canal 1';

% Create Canal2Label
app.Canal2Label = uilabel(app.UIFigure);
app.Canal2Label.Position = [282 270 47 22];
app.Canal2Label.Text = 'Canal 2';

% Create Conectar
app.Conectar = uibutton(app.UIFigure, 'push');
app.Conectar.ButtonPushedFcn = createCallbackFcn(app, @ConectarButtonPushed, true);
app.Conectar.FontSize = 16;
app.Conectar.FontWeight = 'bold';
app.Conectar.Position = [390 15 105 27];
app.Conectar.Text = 'CONECTAR';

% Create Salir
app.Salir = uibutton(app.UIFigure, 'push');
app.Salir.ButtonPushedFcn = createCallbackFcn(app, @SalirButtonPushed, true);
app.Salir.FontSize = 16;
app.Salir.FontWeight = 'bold';
app.Salir.Position = [784 15 100 27];
app.Salir.Text = 'SALIR';

% Create ActividadLampLabel
app.ActividadLampLabel = uilabel(app.UIFigure);
app.ActividadLampLabel.HorizontalAlignment = 'right';
app.ActividadLampLabel.FontSize = 16;
app.ActividadLampLabel.FontAngle = 'italic';
app.ActividadLampLabel.Position = [133 22 70 22];
app.ActividadLampLabel.Text = 'Actividad';

% Create LED_Actividad
app.LED_Actividad = uilamp(app.UIFigure);
app.LED_Actividad.Position = [218 22 20 20];
app.LED_Actividad.Color = [1 0 0];

% Create PARASIMULACINDEFUNCIONESDETRANSFERENCIALabel
app.PARASIMULACINDEFUNCIONESDETRANSFERENCIALabel = uilabel(app.UIFigure);
app.PARASIMULACINDEFUNCIONESDETRANSFERENCIALabel.FontSize = 14;
app.PARASIMULACINDEFUNCIONESDETRANSFERENCIALabel.FontWeight = 'bold';
app.PARASIMULACINDEFUNCIONESDETRANSFERENCIALabel.Position = [434 546 400 22];
app.PARASIMULACINDEFUNCIONESDETRANSFERENCIALabel.Text = 'PARA SIMULACIÓN DE FUNCIONES DE TRANSFERENCIA';

% Show the figure after all components are created
app.UIFigure.Visible = 'on';
end
end

```

```

% App creation and deletion
methods (Access = public)

    % Construct app
    function app = Scope_2Channel_UNAM

        % Create UIFigure and components
        createComponents(app)

        % Register the app with App Designer
        registerApp(app, app.UIFigure)

        % Execute the startup function
        runStartupFcn(app, @startupFcn)

        if nargin == 0
            clear app
        end
    end

    % Code that executes before app deletion
    function delete(app)

        % Delete UIFigure when app is deleted
        delete(app.UIFigure)
    end
end
end
end

```